Efficient Dataset Condensation for Time-Series Forecasting via Target Projection

Liwei Deng
Aalborg University
Aalborg, Denmark
lide@cs.aau.dk

Yan Zhao
University of Electronic Science and Technology of China✉
Chengdu, China
zhaoyan@uestc.edu.cn

Yupu Zhang
University of Electronic Science and Technology of China
Chengdu, China
zhangyupu@std.uestc.edu.cn

Tianfu Wang
The Hong Kong University of Science and Technology (Guangzhou)
Guangzhou, China
tianfuwang.cs@gmail.com

Torben Bach Pedersen
Aalborg University
Aalborg, Denmark
tbp@cs.aau.dk

Kai Zheng*
University of Electronic Science and Technology of China✉
Chengdu, China
zhengkai@uestc.edu.cn

ABSTRACT

Dataset condensation aims to reduce training-data storage and repeated model-training costs by constructing compact training sets that enable predictive performance comparable to full-data training. For time-series forecasting, a key challenge is to represent recurring historical–future relationships using very few training windows. In addition, the cost of constructing these compact sets must remain low enough to preserve the computational savings from subsequent model training. In this study, we propose Projected-Target Coresets (PTC), a framework that pairs real historical windows with targets estimated from the complete training set to remove projection residuals. Its default implementation combines linear target projection with regularized leverage-score selection, enabling closed-form construction without iterative input synthesis or repeated student training. Our theoretical analysis characterizes budget-dependent variation in residual gradient contributions and provides a conditional prediction-error guarantee for linear students. Experiments on eight forecasting benchmarks demonstrate strong forecasting performance across three student architectures, approaching full-data performance in many settings. These results are achieved with substantially lower construction time and memory than iterative methods, alongside reduced downstream training time.

PVLDB Reference Format:
Liwei Deng, Yan Zhao, Yupu Zhang, Tianfu Wang, Torben Bach Pedersen, and Kai Zheng. Efficient Dataset Condensation for Time-Series Forecasting via Target Projection . PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/LIWEIDENG0830/PTC.

*✉ The corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Time-series forecasting is a fundamental component of data-driven systems [10] in domains such as energy management [17, 18], transportation [9, 11], weather monitoring [1, 22], and financial analysis [6, 50]. Recent forecasting models have achieved substantial accuracy improvements by learning from increasingly large collections of multivariate observations [41]. These gains, however, come with considerable computational cost [15, 16, 21]. Architecture selection, hyperparameter tuning, and evaluation across multiple prediction horizons in model development further increase this computational burden [35, 42–44]. The resulting cost makes the construction and reuse of compact training datasets an important data-management concern. Dataset condensation offers a possible remedy by replacing the original training set with a much smaller dataset that preserves its utility for downstream model training [40, 49]. A successful condensed dataset should enable models trained from scratch to approach the performance obtained from the full data while requiring only a fraction of the training time and storage [7].

Most existing condensation methods construct a compact training set by optimizing synthetic data through gradient matching [47, 49], feature-distribution matching [39, 48], kernel-based objectives [28, 52], or training-trajectory matching [4, 13]. Recent studies extend these approaches to time-series forecasting. For example, TimeDC [26] optimizes condensed sequences through frequency-domain and trajectory matching to preserve temporal patterns and learning behavior. CondTSF [7] uses an underlying condensation method to jointly optimize historical inputs and future targets, and periodically corrects the targets using predictions from a model trained on the complete dataset. These methods demonstrate the potential of dataset condensation for forecasting, but constructing an effective compact training set at a practical computational cost still presents two challenges.

C1. Representing recurring forecasting relationships with few observations. Similar historical patterns can be followed by different future observations, reflecting both recurring temporal behavior and variation specific to individual occurrences. For example, repeated traffic patterns may have different subsequent measurements when a temporary disruption occurs [31]. A large training

set provides multiple observations of such patterns, whereas a condensed set may contain only one or a few, giving their particular outcomes a much larger role in training. This requires supervision that reflects the complete training dataset without being dominated by the particular future observations attached to a few retained windows. TimeDC [26] implicitly addresses this requirement by aligning student-training trajectories with those of full-data experts and using frequency matching to preserve temporal structure. However, its forecasting implementation optimizes synthetic historical inputs without updating their associated future targets. The optimization thus adjusts inputs to the given targets, leaving observation-specific variation unchanged at the target level.

C2. Controlling the upfront cost of condensation. Constructing a compact dataset can require substantial computation before downstream training begins. Gradient matching [47, 49] involves model-dependent gradient computations, while trajectory matching [4, 13] additionally requires pretrained experts. For trajectory-based methods, expert parameters must be compatible with those of the student used during condensation [4], making expert preparation dependent on the chosen architecture. Even with a lightweight backbone, preparing the 40 DLinear experts used by our trajectory-based baselines on Traffic at $H = 96$ takes approximately 5.95 GPU hours on T4 GPUs, excluding subsequent condensation. For reference, a full-data PatchTST training run on Traffic at $L = H = 96$ takes a median of 9.82 hours on a T4 GPU in our experiments (see Section 4.6). When construction requires multiple full-data training runs followed by iterative data optimization, the initial computation can outweigh the savings from subsequent training on fewer examples. Reducing this upfront cost is therefore necessary for condensation to provide computational benefits within a practical number of downstream training runs.

To address these challenges, we propose Projected-Target Coresets (PTC), a framework that combines projected-target construction with leverage-based historical-window selection. Specifically, in its default implementation, a linear operator is fitted on the complete training set to estimate the relationship between historical inputs and future observations. Its fitted outputs separate the future variation represented by this relationship from the remaining residuals. We retain real historical windows and replace their observed futures with these projected targets, allowing their supervision to reflect information aggregated across the training set rather than only individual future observations (addressing **C1**). To allocate the limited window budget, regularized leverage scores quantify the contributions of candidate histories to the same projection space and determine their sampling probabilities. The selected histories and projected targets form a compact dataset in the standard input–target format. In this implementation, both target construction and window scoring use closed-form matrix operations, avoiding iterative input synthesis and repeated student training (addressing **C2**). The resulting dataset can be reused across forecasting architectures without changing their training procedures. Our theoretical analysis characterizes budget-dependent variation in residual gradient contributions and establishes that target projection does not increase the expected prediction error of linear least-squares students under suitable assumptions.

We evaluate PTC on eight multivariate forecasting benchmarks against random selection and eight condensation baselines. With DLinear students, PTC achieves the best forecasting performance across the evaluated settings, including a 20.6% MSE reduction over the strongest baseline on ETTm1 with only ten retained windows, and approaches full-data performance on most benchmarks. These gains largely persist when the same condensed datasets are used to train iTransformer and PatchTST students. To further examine the contribution of target projection, we replace the targets of an iterative baseline with PTC target projection and observe improved forecasting performance in most evaluated settings. Beyond forecasting performance, the efficiency evaluation demonstrates low dataset condensation costs and substantial downstream training savings. For example, on Traffic at $L = H = 96$ and $K = 25$, PTC constructs the condensed dataset in few seconds and provides a 6.7× speedup in cumulative DLinear training time, with forecasting accuracy comparable to full-data training.

The main contributions of this paper are summarized as follows.

- We analyze the effect of target residuals on learning from condensed forecasting datasets. The analysis characterizes budget-dependent variation in residual gradient contributions and establishes a conditional prediction-error guarantee for linear students.
- We propose Projected-Target Coresets (PTC), a framework that pairs selected real historical windows with predictable future components estimated from the complete training set. A shared projection space supports both target construction and window selection without model-dependent iterative optimization.
- We conduct extensive experiments on eight multivariate forecasting benchmarks. The results show that PTC consistently outperforms representative condensation methods, and transfers effectively across multiple downstream students. The efficiency evaluation further demonstrates substantially lower dataset construction and downstream model training time.

The remainder of this paper is organized as follows. Section 2 introduces the preliminaries and defines the problem addressed. Section 3 presents the proposed PTC framework, together with its theoretical properties and computational complexity. Section 4 reports on extensive experimental evaluations of forecasting performance, cross-architecture transferability, and efficiency. Section 5 reviews related work, and Section 6 concludes and discusses future research directions.

2 PRELIMINARIES

We introduce concepts important to time-series forecasting and dataset condensation, and then define the problem addressed.

Definition 1 (Multivariate Time Series). *A multivariate time series* $\mathbf{Z} = \langle \mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T \rangle$ *is a time-ordered sequence of* T *observations, where each observation* $\mathbf{z}_t \in \mathbb{R}^C$ *contains* C *variables. The time series is univariate when* $C = 1$ *and multivariate when* $C > 1$.

Time-series forecasting uses historical observations to predict their subsequent future. We formalize the basic data unit used for model training as follows.

Definition 2 (Forecasting Instance). *Given a multivariate time series* $\mathbf{Z}$*, an input length* L*, and a prediction horizon* H*, a forecasting instance at time step* t *is a pair* $(\mathbf{X}_t, \mathbf{Y}_t)$*. The historical input* $\mathbf{X}_t = \langle \mathbf{z}_{t-L+1}, \ldots, \mathbf{z}_t \rangle \in \mathbb{R}^{L \times C}$ *contains the preceding* L *observations,*

and the forecasting target $\mathbf{Y}_t = \langle \mathbf{z}_{t+1}, \ldots, \mathbf{z}_{t+H} \rangle \in \mathbb{R}^{H \times C}$ *contains the subsequent* H *observations.*

A rolling-window procedure generates forecasting instances at multiple time steps, which together form a forecasting dataset.

Definition 3 (Forecasting Dataset). *A forecasting dataset* $\mathcal{D} = \{(\mathbf{X}_i, \mathbf{Y}_i)\}_{i=1}^{N}$ *is a collection of* N *forecasting instances constructed from one or more time series, where* $(\mathbf{X}_i, \mathbf{Y}_i)$ *denotes the* i*-th instance. Each historical input* $\mathbf{X}_i$ *and forecasting target* $\mathbf{Y}_i$ *follow the dimensions specified in Definition 2. Forecasting instances generated using a sliding window may contain overlapping observations.*

Let $\mathcal{F}$ denote a class of forecasting models, where each $f \in \mathcal{F}$ maps a historical input in $\mathbb{R}^{L \times C}$ to an H-step forecast in $\mathbb{R}^{H \times C}$. Given a forecasting loss ℓ, the empirical loss of f on $\mathcal{D}$ is defined as follows:

$$\mathcal{L}_{\mathcal{D}}(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\mathbf{X}_i), \mathbf{Y}_i) \,. \tag{1}$$

We use $f_{\mathcal{D}} = \mathrm{Train}(\mathcal{D})$ to denote a forecasting model obtained by training on $\mathcal{D}$. Let $\mathcal{P}$ denote the distribution of forecasting instances. The predictive error of $f_{\mathcal{D}}$ on unseen forecasting instances is as follows:

$$\mathcal{E}_{\mathcal{P}}(f_{\mathcal{D}}) = \mathbb{E}_{(\mathbf{X},\mathbf{Y})\sim\mathcal{P}} \left[\ell(f_{\mathcal{D}}(\mathbf{X}), \mathbf{Y}) \right] . \tag{2}$$

A lower predictive error indicates better forecasting performance.

Dataset condensation aims to replace $\mathcal{D}$ with a substantially smaller dataset that follows the same input-target format.

Definition 4 (Condensed Forecasting Dataset). *Given a forecasting dataset* $\mathcal{D}$ *containing* N *instances and a condensation budget* $K \ll N$*, a condensed forecasting dataset is a set* $\mathcal{S} = \{(\widetilde{\mathbf{X}}_j, \widetilde{\mathbf{Y}}_j)\}_{j=1}^{K}$ *derived from* $\mathcal{D}$*. Each condensed input* $\widetilde{\mathbf{X}}_j \in \mathbb{R}^{L \times C}$ *and condensed target* $\widetilde{\mathbf{Y}}_j \in \mathbb{R}^{H \times C}$ *has the same form as an ordinary forecasting instance. The condensed inputs and targets may be selected from the original dataset or constructed using information contained in the original dataset.*

Although Definition 4 permits both components of a condensed instance to be selected or constructed, existing methods have largely learned condensed sequences through iterative optimization [28, 52]. Methods adapted from image dataset condensation typically treat synthetic samples as learnable variables and update them using model-dependent matching objectives, while the corresponding labels or targets are often fixed [4, 48, 49]. The forecasting implementation of TimeDC follows this input-centered design by iteratively optimizing the condensed input sequences while leaving their future targets unchanged after initialization [26]. CondTSF further divides each synthetic sequence into historical and future segments and periodically corrects the latter using an expert model, yet the complete sequence remains iteratively optimized by an underlying condensation backbone [7]. Thus, even methods that modify $\widetilde{\mathbf{Y}}$ generally retain iterative synthesis of $\widetilde{\mathbf{X}}$ as their principal condensation mechanism.

Regardless of how its inputs and targets are obtained, a condensed forecasting dataset should enable a model trained on it to achieve predictive performance comparable to that obtained from the original dataset.

Definition 5 (Dataset Condensation for Time-Series Forecasting). *Given an original forecasting dataset* $\mathcal{D}$ *and a condensation budget* $K \ll N$*, dataset condensation for time-series forecasting aims to construct a condensed forecasting dataset* $\mathcal{S}$ *with* $|\mathcal{S}| = K$ *such that a forecasting model* $f_{\mathcal{S}} = \mathrm{Train}(\mathcal{S})$ *achieves predictive performance comparable to that of a model* $f_{\mathcal{D}} = \mathrm{Train}(\mathcal{D})$ *trained on the original dataset, which can be expressed as follows:*

$$\mathcal{E}_{\mathcal{P}}(f_{\mathcal{S}}) \simeq \mathcal{E}_{\mathcal{P}}(f_{\mathcal{D}}). \tag{3}$$

Definition 5 specifies the desired predictive utility of $\mathcal{S}$ without prescribing how its inputs and targets must be constructed. A common solution treats the condensed dataset as a collection of learnable variables. Since the predictive error on the unknown distribution $\mathcal{P}$ is unavailable during construction, the empirical loss on the original training dataset is commonly used as a surrogate. This yields the following bi-level formulation:

$$\min_{\mathcal{S}} \mathcal{L}_{\mathcal{D}}(f_{\mathcal{S}}) \quad \text{s.t.} \quad f_{\mathcal{S}} \in \arg\min_{f \in \mathcal{F}} \mathcal{L}_{\mathcal{S}}(f), \qquad |\mathcal{S}| = K. \tag{4}$$

The inner optimization trains a forecasting model on the current condensed dataset, while the outer optimization updates the condensed data according to the resulting model's performance on the original dataset. Directly solving Equation (4) requires repeatedly optimizing the inner model and differentiating its outcome with respect to the condensed data. Existing methods approximate this process by matching gradients [49], feature distributions [48], kernels [28, 52], frequencies [26], or parameter trajectories [4, 13] between the original and condensed datasets. Although these objectives differ, most of them still require repeated model-dependent updates of the condensed inputs. This nested model and data optimization is consequently a major source of construction time and memory consumption.

This computational burden motivates an alternative construction strategy that avoids nested optimization. The forecasting condensation problem characterizes the desired dataset in terms of its cardinality and predictive utility, leaving the construction procedure unspecified. The method developed in this paper directly constructs the condensed inputs and targets from the original training dataset, thereby bypassing repeated inner-model training and outer synthetic-input optimization.

3 METHODOLOGY

We proceed to detail the proposed framework PTC and its default linear implementation, followed by the theoretical and complexity analyses.

3.1 Framework Overview

We propose Projected-Target Coresets (PTC) framework, which selects real historical windows and constructs their targets from the complete training dataset, replacing the iterative input synthesis illustrated in Figure 1(a). Given an original forecasting dataset $\mathcal{D} = \{(\mathbf{X}_i, \mathbf{Y}_i)\}_{i=1}^{N}$ and a condensation budget K, let $\mathcal{I} \subseteq \{1, \ldots, N\}$ denote the selected indices with $|\mathcal{I}| = K$, and let $g_{\mathcal{D}} : \mathbb{R}^{L \times C} \to \mathbb{R}^{H \times C}$ denote a target generator fitted exclusively on $\mathcal{D}$. The condensed dataset takes the form as follows:

$$\mathcal{S}(\mathcal{I}, g_{\mathcal{D}}) = \{(\mathbf{X}_i, g_{\mathcal{D}}(\mathbf{X}_i)) \mid i \in \mathcal{I}\} \,. \tag{5}$$

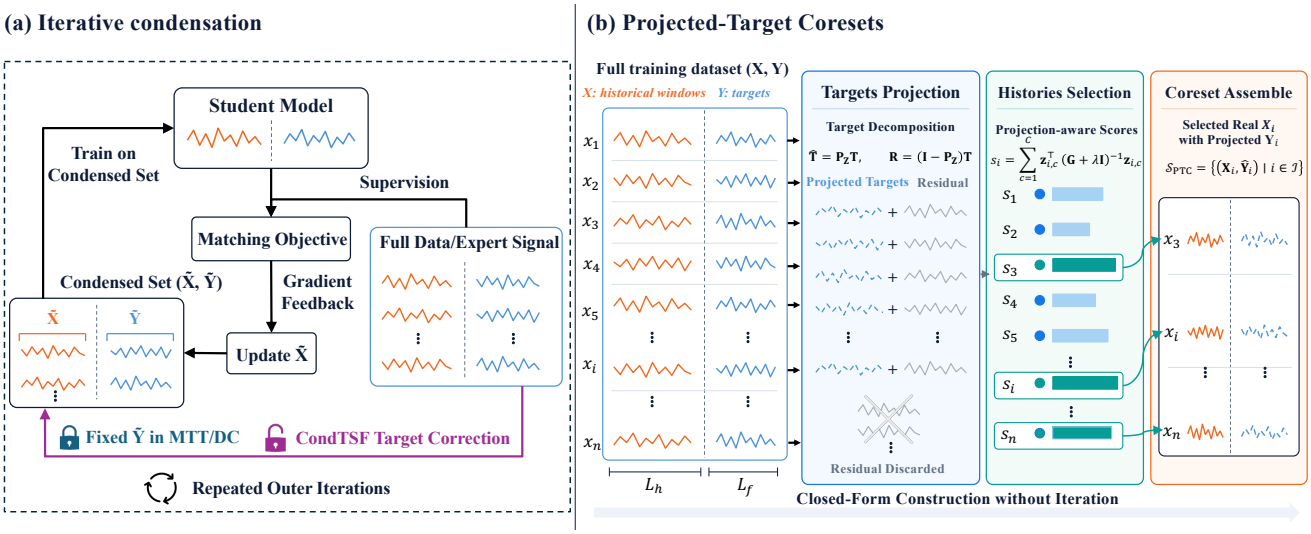


Figure 1: Comparison between iterative time-series dataset condensation and the proposed Projected-Target Coresets framework. Existing methods repeatedly optimize synthetic data through model training and matching objectives, whereas our method constructs the condensed dataset directly by projecting future targets and selecting informative historical windows.

The target generator estimates the forecasting relationship between historical inputs and future targets using the complete training dataset. Its outputs provide supervision for the selected historical windows, allowing their targets to incorporate information beyond the individual observed futures. The generator is used only during dataset construction and need not share the architecture of the downstream forecasting model. Once $\mathcal{S}$ has been constructed, a student model can be trained from scratch using the standard forecasting loss.

Specifically, the construction combines target projection and historical-window selection in a shared representation space, as illustrated in Figure 1(b). A lightweight channel-independent linear operator fitted on all training windows decomposes the observed futures into a component represented by the fitted linear relationship and an orthogonal residual. The projected component serves as the constructed targets, while the residual is discarded. Projection-aware importance scores are computed in the same space to sample K distinct historical windows without replacement. The selected windows are then paired with their projected targets to assemble the condensed dataset. Both components use closed-form matrix operations on the original training data. The resulting construction bypasses the conventional bi-level procedure in Equation (4), without repeated inner-student training, gradient-based updates of historical inputs, or expert-trajectory preparation. The following subsections detail projected-target construction and leverage-based historical-window selection, followed by the theoretical and complexity analyses.

3.2 Projected-Target Construction

When only a few historical windows are retained in dataset condensation, their observed futures may place disproportionate emphasis on sample-specific residuals. To reduce this influence, target construction uses the forecasting relationship estimated from the complete training dataset to provide supervision for the retained windows. In our default implementation, we approximate this relationship with a linear operator, whose fitted outputs serve as the constructed targets. This formulation yields a closed-form estimate of the future component represented by the linear operator.

For each instance $(\mathbf{X}_i, \mathbf{Y}_i)$ and channel c, let $\mathbf{x}_{i,c} \in \mathbb{R}^L$ and $\mathbf{y}_{i,c} \in \mathbb{R}^H$ denote the corresponding historical and future sequences. We represent the history by the intercept-augmented vector $\mathbf{z}_{i,c} = [\mathbf{x}_{i,c}^\top, 1]^\top \in \mathbb{R}^d$, where $d = L + 1$, and collect all channel–window pairs into a shared design matrix $\mathbf{Z}$ and target matrix $\mathbf{T}$:

$$\mathbf{Z} = \begin{bmatrix} \mathbf{z}_{1,1}^\top \\ \vdots \\ \mathbf{z}_{N,C}^\top \end{bmatrix} \in \mathbb{R}^{NC \times d}, \qquad \mathbf{T} = \begin{bmatrix} \mathbf{y}_{1,1}^\top \\ \vdots \\ \mathbf{y}_{N,C}^\top \end{bmatrix} \in \mathbb{R}^{NC \times H}. \tag{6}$$

This representation treats each channel sequence as an observation of the same temporal forecasting relation. It pools information from NC channel–window pairs while keeping the number of fitted parameters independent of the number of channels, making the construction applicable to high-dimensional multivariate datasets.

We estimate the shared temporal operator by minimizing the squared error between the fitted and observed futures over all channel–window pairs. Let $\widehat{\boldsymbol{\Theta}} \in \mathbb{R}^{d \times H}$ denote the fitted operator and $\mathbf{P}_Z$ the orthogonal projector onto the column space of $\mathbf{Z}$. The multi-output least-squares solution and its fitted targets satisfy:

$$\widehat{\boldsymbol{\Theta}} = \arg\min_{\boldsymbol{\Theta} \in \mathbb{R}^{d \times H}} \|\mathbf{Z}\boldsymbol{\Theta} - \mathbf{T}\|_F^2 = \left(\mathbf{Z}^\top \mathbf{Z}\right)^\dagger \mathbf{Z}^\top \mathbf{T}, \qquad \widehat{\mathbf{T}} = \mathbf{Z}\widehat{\boldsymbol{\Theta}} = \mathbf{P}_Z \mathbf{T}, \tag{7}$$

where $\mathbf{P}_Z = \mathbf{Z}(\mathbf{Z}^\top\mathbf{Z})^\dagger\mathbf{Z}^\top$ and $(\cdot)^\dagger$ denotes the pseudoinverse. Each column of $\widehat{\mathbf{T}}$ is the projection of the corresponding column of $\mathbf{T}$ onto the column space of $\mathbf{Z}$, giving the decomposition as follows:

$$\mathbf{T} = \widehat{\mathbf{T}} + \mathbf{R} = \mathbf{P}_Z \mathbf{T} + (\mathbf{I} - \mathbf{P}_Z)\,\mathbf{T}, \qquad \mathbf{Z}^\top \mathbf{R} = \mathbf{0}. \tag{8}$$

The projected component $\widehat{\mathbf{T}}$ captures the future variation represented by the fitted linear relationship, whereas $\mathbf{R} = (\mathbf{I} - \mathbf{P}_Z)\mathbf{T}$ contains the remaining variation. Orthogonality to $\mathbf{Z}$ does not imply

that $\mathbf{R}$ is unpredictable by other models. It may contain nonlinear structure or systematic changes that the fitted relationship does not capture, so discarding it can remove useful forecasting information. Using projected targets is intended to limit the influence of individual residual realizations under small condensation budgets, with the resulting benefit depending on the dataset and downstream student.

For a historical window $\mathbf{X}_i$, the rows of $\widehat{\mathbf{T}}$ associated with its channels are reassembled into a projected future $\widehat{\mathbf{Y}}_i \in \mathbb{R}^{H\times C}$ according to the following equation:

$$\widehat{\mathbf{Y}}_{i,:,c} = \widehat{\boldsymbol{\Theta}}^{\top}\mathbf{z}_{i,c}, \qquad c = 1,\dots,C. \tag{9}$$

Although $\widehat{\mathbf{Y}}_i$ is paired with the real historical window $\mathbf{X}_i$, it is determined by an operator estimated from all training instances. Each condensed target therefore carries information about the dataset-level forecasting relationship rather than only the single observed future originally following that window.

The linear projection provides a lightweight instantiation of the target generator $g_{\mathcal{D}}$ in Equation (5), with a closed-form solution that remains independent of the downstream student architecture. More expressive forecasting estimators can also be used to construct targets, but require an additional training process. As shown in Section 4, their greater expressiveness does not consistently translate into better downstream forecasting performance, and the relative effectiveness of target generators varies across datasets and student models. The linear projection therefore serves as an efficient default rather than a uniformly superior estimator. Having defined the projected targets, we next consider how to select historical windows using the same historical representation.

3.3 Leverage-Based Historical-Window Selection

After constructing the projected targets, the remaining problem is to identify a small set of historical windows using the representation from which these targets are derived. Uniform sampling treats all windows equally, although forecasting datasets commonly contain many overlapping and highly redundant windows. Thus, a suitable selection criterion should instead distinguish between windows that repeat common temporal patterns and those that contain historical variation less represented by other training instances.

The geometry relevant to this distinction is encoded by the design matrix $\mathbf{Z}$ used in Equation (6). Its Gram matrix $\mathbf{G} = \mathbf{Z}^{\top}\mathbf{Z}$ summarizes the historical information represented across the complete training dataset. Since $\mathbf{G} = \sum_{i,c}\mathbf{z}_{i,c}\mathbf{z}_{i,c}^{\top}$, repeated histories contribute repeatedly to this aggregate. The regularized inverse-Gram score evaluates each history relative to the complete collection of historical observations. Holding the other observations fixed, adding copies of the same history lowers the score assigned to each copy. This gives each repeated history less sampling weight as its information becomes more extensively represented in the dataset.

Based on this observation, we define the leverage-based importance score of a historical window $\mathbf{X}_i$ as follows:

$$s_i = \sum_{c=1}^{C}\mathbf{z}_{i,c}^{\top}\left(\mathbf{G}+\lambda\mathbf{I}\right)^{-1}\mathbf{z}_{i,c}, \tag{10}$$

Algorithm 1: PTC with Linear Target Projection

Input: training dataset $\mathcal{D}$; condensation budget K; regularization coefficient λ
Output: condensed dataset $\mathcal{S}_{\mathrm{PTC}}$

1 $\mathbf{G} \leftarrow \sum_{i,c}\mathbf{z}_{i,c}\mathbf{z}_{i,c}^{\top}$; $\mathbf{B} \leftarrow \sum_{i,c}\mathbf{z}_{i,c}\mathbf{y}_{i,c}^{\top}$;
2 $\widehat{\boldsymbol{\Theta}} \leftarrow \mathbf{G}^{\dagger}\mathbf{B}$; $\mathbf{A} \leftarrow (\mathbf{G}+\lambda\mathbf{I})^{-1}$;
3 **for** $i \leftarrow 1$ **to** N **do**
4 $s_i \leftarrow \sum_{c=1}^{C}\mathbf{z}_{i,c}^{\top}\mathbf{A}\mathbf{z}_{i,c}$;
5 $p_i \leftarrow s_i / \sum_{j=1}^{N} s_j$ for each $i \in \{1,\dots,N\}$;
6 Draw K distinct indices $\mathcal{I}$ without replacement according to $\{p_i\}_{i=1}^{N}$;
7 **foreach** $i \in \mathcal{I}$ **do**
8 $\widehat{\mathbf{Y}}_{i,:,c} \leftarrow \widehat{\boldsymbol{\Theta}}^{\top}\mathbf{z}_{i,c}$ for each $c \in \{1,\dots,C\}$;
9 **return** $\mathcal{S}_{\mathrm{PTC}} \leftarrow \{(\mathbf{X}_i, \widehat{\mathbf{Y}}_i) \mid i \in \mathcal{I}\}$;

where $\lambda > 0$ stabilizes the inverse when the historical representations are highly correlated or rank deficient. Each term in Equation (10) is a regularized statistical leverage score, a standard measure of the contribution of a matrix row to its underlying linear subspace [5, 8]. The channel-wise scores are aggregated because the unit retained by the condensed dataset is a complete multivariate window rather than an individual channel sequence.

For illustration, consider four univariate windows of length two, three with history $(1,1)^{\top}$ and one with history $(1,2)^{\top}$. Including the intercept gives representations $\mathbf{a} = (1,1,1)^{\top}$ and $\mathbf{b} = (1,2,1)^{\top}$, so $\mathbf{G} = 3\mathbf{a}\mathbf{a}^{\top} + \mathbf{b}\mathbf{b}^{\top}$. As λ approaches zero, the score of each repeated history approaches $1/3$, while the score of the distinct history approaches 1. Thus, the repeated histories share their contribution, and the distinct history receives greater sampling weight.

The scores can be defined as a sampling distribution over the original historical windows as follows:

$$p_i = \frac{s_i}{\sum_{j=1}^{N} s_j}, \qquad i = 1,\dots,N. \tag{11}$$

We draw K distinct indices without replacement according to this distribution and denote the resulting index set by $\mathcal{I}$. The condensed dataset is then constructed as follows:

$$\mathcal{S}_{\mathrm{PTC}} = \left\{\left(\mathbf{X}_i, \widehat{\mathbf{Y}}_i\right) \mid i \in \mathcal{I}\right\}, \tag{12}$$

where $\widehat{\mathbf{Y}}_i$ is the projected target defined in Equation (9). Historical-window selection and target construction are linked through the same design matrix $\mathbf{Z}$. The scores determine which real histories are retained, while their projected targets encode the forecasting relationship estimated from the complete training dataset.

Algorithm 1 summarizes how projected-target construction and historical-window selection are combined. Lines 1–2 accumulate the matrices $\mathbf{G} = \mathbf{Z}^{\top}\mathbf{Z}$ and $\mathbf{B} = \mathbf{Z}^{\top}\mathbf{T}$ and compute the shared forecasting operator $\widehat{\boldsymbol{\Theta}}$ and the regularized inverse $\mathbf{A}$. Lines 3–6 evaluate the contribution of each historical window to the projection space, normalize the resulting scores into a sampling distribution, and select K distinct windows. Lines 7–9 generate projected targets for the selected windows and assemble the final condensed dataset. Thus, although the projection is expressed using the complete matrices $\mathbf{Z}$ and $\mathbf{T}$, neither matrix nor the full projected target matrix needs to be explicitly materialized during construction.

3.4 Theoretical Analysis

We analyze the default linear target projection through its effect on training gradients, using the complete training dataset as a reference. The decomposition in Equation (8) gives $\mathbf{T} = \widehat{\mathbf{T}} + \mathbf{R}$ with $\mathbf{Z}^\top \mathbf{R} = \mathbf{0}$. For a channel-independent linear student that predicts each channel as $\boldsymbol{\Theta}^\top \mathbf{z}_{i,c}$ with parameters shared across channels, this orthogonality makes the residual contribution to the full-data training gradient zero. The corresponding contribution can be nonzero after retaining only K windows. We therefore examine how condensation changes this residual contribution by comparing observed and projected targets on the same retained inputs.

Let $f_{\boldsymbol{\theta}}$ denote a student with parameters $\boldsymbol{\theta}$, and let $\mathbf{R}_i = \mathbf{Y}_i - \widehat{\mathbf{Y}}_i$ denote the projection residual associated with window i. For a retained index set $\mathcal{I}$ of size K, define the training losses as follows:

$$\begin{aligned} \mathcal{L}_{\mathrm{obs},\mathcal{I}}(\boldsymbol{\theta}) &= \frac{1}{KCH} \sum_{i \in \mathcal{I}} \|f_{\boldsymbol{\theta}}(\mathbf{X}_i) - \mathbf{Y}_i\|_F^2, \\ \mathcal{L}_{\mathrm{proj},\mathcal{I}}(\boldsymbol{\theta}) &= \frac{1}{KCH} \sum_{i \in \mathcal{I}} \left\| f_{\boldsymbol{\theta}}(\mathbf{X}_i) - \widehat{\mathbf{Y}}_i \right\|_F^2. \end{aligned} \tag{13}$$

Let $\mathbf{J}_i(\boldsymbol{\theta})$ be the Jacobian of $\mathrm{vec}(f_{\boldsymbol{\theta}}(\mathbf{X}_i))$ with respect to $\boldsymbol{\theta}$. The following result expresses the gradient difference between the two losses directly through the residuals removed by PTC.

Proposition 3.1 (Residual contributions to subset gradients). *For fixed training data and student parameters, the gradient difference induced by the projection residual of window i is:*

$$\mathbf{d}_i = -\frac{2}{CH} \mathbf{J}_i(\boldsymbol{\theta})^\top \mathrm{vec}(\mathbf{R}_i). \tag{14}$$

Averaging these contributions over the retained windows gives the gradient difference between training with observed and projected targets as follows:

$$\boldsymbol{\Delta}_{\mathcal{I}} := \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{obs},\mathcal{I}} - \nabla_{\boldsymbol{\theta}} \mathcal{L}_{\mathrm{proj},\mathcal{I}} = \frac{1}{K} \sum_{i \in \mathcal{I}} \mathbf{d}_i. \tag{15}$$

Let $\boldsymbol{\Delta}_{\mathrm{full}} = N^{-1} \sum_{i=1}^{N} \mathbf{d}_i$ denote the corresponding gradient difference on the complete dataset. If $\mathcal{I}$ contains K windows sampled uniformly without replacement from $N > 1$ training windows, then we have:

$$\begin{aligned} \mathbb{E}_{\mathcal{I}}[\boldsymbol{\Delta}_{\mathcal{I}}] &= \boldsymbol{\Delta}_{\mathrm{full}}, \\ \mathbb{E}_{\mathcal{I}}\left[\|\boldsymbol{\Delta}_{\mathcal{I}} - \boldsymbol{\Delta}_{\mathrm{full}}\|_2^2\right] &= \frac{N-K}{K(N-1)} V_{\mathbf{d}}, \end{aligned} \tag{16}$$

where $V_{\mathbf{d}} = N^{-1} \sum_{i=1}^{N} \|\mathbf{d}_i - \boldsymbol{\Delta}_{\mathrm{full}}\|_2^2$ measures the variation of the residual gradient contributions across individual windows.

Proof. Please see Appendix A.1 in the artifact repository for detailed proof. □

Proposition 3.1 relates the condensation budget to the deviation of subset residual contributions from their full-data counterpart. With the training data and student parameters fixed, $V_{\mathbf{d}}$ is computed over all N windows and does not depend on K. The factor $(N-K)/(K(N-1))$ therefore determines how the expected squared deviation changes with the budget.

For the channel-independent linear student with parameters shared across channels, the full-data gradient difference satisfies:

$$\nabla_{\boldsymbol{\Theta}} \mathcal{L}_{\mathrm{obs,full}} - \nabla_{\boldsymbol{\Theta}} \mathcal{L}_{\mathrm{proj,full}} = -\frac{2}{NCH} \mathbf{Z}^\top \mathbf{R} = \mathbf{0}. \tag{17}$$

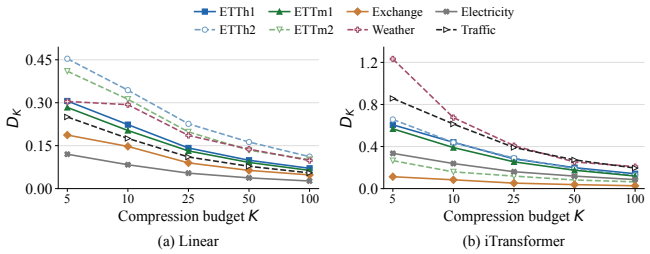


Figure 2: Residual gradient deviation D_K across condensation budgets for (a) a channel-independent linear student and (b) iTransformer.

Thus, $\boldsymbol{\Delta}_{\mathrm{full}} = \mathbf{0}$, whereas the subset contribution is generally nonzero. Proposition 3.1 then gives:

$$\mathbb{E}_{\mathcal{I}}\left[\|\boldsymbol{\Delta}_{\mathcal{I}}\|_2^2\right] = \frac{N-K}{K(N-1)} V_{\mathbf{d}}. \tag{18}$$

Whenever $V_{\mathbf{d}} > 0$, smaller budgets yield a larger expected squared norm of the residual gradient contribution removed by target projection. This result shows that projection residuals, whose gradient contribution is zero on the complete dataset, can produce nonzero contributions on retained subsets that PTC removes through target projection.

Figure 2 examines residual gradient contributions on the benchmark datasets at $L = H = 96$. We measure their deviation from the full-data counterpart by $D_K = \sqrt{\mathbb{E}_{\mathcal{I}}[\|\boldsymbol{\Delta}_{\mathcal{I}} - \boldsymbol{\Delta}_{\mathrm{full}}\|_2^2]}$, estimated using uniformly sampled subsets with student parameters held fixed. Each point for the linear student uses 200 subsets. For iTransformer, we use three random initializations and the same 50 subsets per budget across initializations, averaging the squared deviations before taking the square root. Both panels show larger D_K at smaller budgets, consistent with Proposition 3.1. For the linear student, $\boldsymbol{\Delta}_{\mathrm{full}} = \mathbf{0}$, so D_K directly measures the root mean squared magnitude of the residual gradient contribution removed by target projection. For iTransformer, the full-data residual contribution is generally nonzero, and D_K measures deviations around that reference. These results show that the budget-dependent variation in residual gradient contributions also occurs beyond the linear case.

The gradient analysis identifies a residual contribution whose variation across uniformly sampled subsets increases at smaller budgets and which target projection removes. To connect this mechanism with downstream learning, Appendix A.2 further establishes that projected targets preserve prediction bias and do not increase the expected prediction error of a linear student under a shared linear conditional mean and uncorrelated residuals with equal total variance. Together, these results provide a mechanism-based explanation and a conditional prediction-error guarantee for target projection, whose practical effectiveness across student architectures is evaluated in Section 4.

3.5 Complexity Analysis

We analyze the construction cost with respect to the number of original forecasting windows N and the condensation budget K, while treating the input length, number of variables, and prediction horizon as fixed specifications of a forecasting task. The target

Table 1: Dataset statistics. Condensed size are calculated at $L = H = 96$.

Dataset	Variables	Timesteps	Interval	Condensed Size (%) $K = 10$	$K = 25$
ETTh1/ETTh2	7	17,420	1 h	0.118	0.296
ETTm1/ETTm2	7	69,680	15 min	0.029	0.073
Exchange	8	7,588	1 day	0.195	0.488
Weather	21	52,696	10 min	0.027	0.068
Electricity	321	26,304	1 h	0.055	0.137
Traffic	862	17,544	1 h	0.083	0.207

projection and projection-aware importance scores are computed directly from the historical inputs and future targets in the training dataset, without iterative optimization. Their computation is therefore linear in N. Sampling K distinct windows from the resulting importance distribution requires $O(N \log K)$ time with a top-K implementation, giving an overall construction complexity of $O(N \log K)$. The matrices used by the projection have dimensions determined by the forecasting configuration and do not grow with the number of training windows. Apart from these fixed-size matrices, storing the window scores requires $O(N)$ auxiliary memory and storing the condensed dataset requires $O(K)$ samples. The auxiliary memory can be reduced to $O(K)$ by maintaining the selected candidates through a streaming top-K procedure.

4 EXPERIMENTS

4.1 Experimental Setup

4.1.1 Datasets. Following prior study on time-series dataset condensation [7, 26], we conduct experiments on eight widely used multivariate forecasting benchmarks.

- **ETT** contains measurements from electricity transformers, including oil temperature and six power-load variables. We use two hourly datasets, ETTh1 and ETTh2, and two 15-minute datasets, ETTm1 and ETTm2.
- **Exchange** contains the daily exchange rates of eight countries collected from 1990 to 2016.
- **Weather** contains 21 meteorological variables, including air temperature and humidity, recorded every 10 minutes at a weather station in Germany.
- **Electricity** records the hourly electricity consumption of 321 clients from 2012 to 2014.
- **Traffic** contains hourly road occupancy measurements collected from 862 sensors on freeways in the San Francisco Bay Area.

Following the standard chronological splits [20, 29, 33, 46, 51], we divide the ETT datasets into training, validation, and test sets with a ratio of 6/2/2, and use a ratio of 7/1/2 for the remaining datasets. Table 1 summarizes dataset statistics.

4.1.2 Baselines. We compare PTC with a random selection baseline and eight representative dataset condensation methods.

- **Rand** randomly selects a prescribed number of training windows and retains their observed targets.
- **DC** [49] optimizes a synthetic dataset by matching the training gradients produced by the synthetic and original datasets.
- **DM** [48] constructs synthetic data by matching the distributions of representations extracted from the original and synthetic datasets.
- **KIP** [28] learns a compact set of inducing points by minimizing a kernel-ridge-regression objective.
- **MTT** [4] optimizes synthetic data such that models trained on it follow parameter trajectories similar to those of models trained on the original dataset.
- **DATM** [13] extends trajectory matching by aligning the difficulty of the matched expert trajectories with the size of the condensed dataset.
- **FRePo** [52] constructs synthetic data through neural feature regression and maintains a pool of models to reduce overfitting during condensation.
- **CondTSF** [7] jointly optimizes historical inputs and future targets as complete sequences using an underlying condensation method, with periodic target correction. We use MTT as its default backbone and denote the DC-based variant by CondTSF†.

We additionally report results obtained by training on the complete training set as a reference.

4.1.3 Evaluation Metrics. We evaluate forecasting accuracy using Mean Squared Error (MSE) and Mean Absolute Error (MAE). Let y_q and $\widehat{y}_q$ denote the ground-truth and predicted values, respectively, and let Q denote the total number of scalar predictions. The metrics are defined as follows:

$$\mathrm{MSE} = \frac{1}{Q}\sum_{q=1}^{Q}(\widehat{y}_q - y_q)^2, \qquad \mathrm{MAE} = \frac{1}{Q}\sum_{q=1}^{Q}|\widehat{y}_q - y_q|. \tag{19}$$

Lower MSE and MAE indicate better forecasting performance. To evaluate computational efficiency, we additionally measure dataset-construction time and peak memory consumption, as well as cumulative downstream student-training time.

4.1.4 Implementation Details. All methods are implemented in Pytorch under the standard multivariate forecasting setting, using an input length of 96, prediction lengths in $\{96, 192, 336, 720\}$, and condensation budgets $K \in \{10, 25\}$. All methods use the same randomly sampled training windows as the initial condensed set under each experimental configuration. DLinear serves as the condensation backbone for all architecture-dependent methods. We use the baseline implementations provided by CondTSF[1]. For trajectory-based methods, we construct 40 expert trajectories for each dataset and prediction length. Each expert is trained for 80 epochs using SGD with a learning rate of 5×10^{-4} and momentum 0.9. Iterative baselines are run for 300 condensation iterations, with 20 inner optimization steps per iteration for methods that unroll student training. For PTC, we set $\lambda = \max\{10^{-8}, 10^{-6}\,\mathrm{tr}(\mathbf{G})/d\}$ to stabilize the Gram-matrix inverse used for window selection. Unless otherwise stated, PTC denotes the default implementation using linear target projection and regularized leverage-score selection. Each condensed dataset is evaluated by training the student from random initialization for at most 500 epochs. We evaluate validation MSE every 10 epochs and stop training after 10 consecutive evaluations without improvement. The checkpoint with the lowest validation MSE is used for test evaluation, and results are averaged over five independent student initializations. The student models follow the

[1] https://github.com/RafaDD/CondTSF

Table 2: Test MSE and MAE of DLinear students. Results are averaged over four prediction horizons {96, 192, 336, 720} and five student seeds. Best and second-best results are shown in bold and underlined. Full Data runs with one seed and is reported as a reference, excluding from the ranking.

Dataset	K	Rand		DC		DM		KIP		MTT		DATM		FRePo		CondTSF		CondTSF†		PTC		Full Data	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	10	0.843	0.633	1.236	0.759	0.846	0.645	0.888	0.667	0.890	0.711	0.795	0.659	0.808	0.623	0.692	0.604	0.590	0.542	**0.492**	**0.482**	0.463	0.456
	25	0.674	0.573	0.699	0.586	0.683	0.579	0.699	0.581	0.849	0.694	0.745	0.639	0.671	0.573	0.663	0.587	0.546	0.516	**0.461**	**0.454**		
ETTh2	10	0.779	0.587	0.825	0.624	0.941	0.634	0.885	0.604	1.081	0.716	0.998	0.687	0.883	0.622	0.732	0.596	0.592	0.535	**0.526**	**0.501**	0.491	0.480
	25	0.687	0.553	0.789	0.607	0.720	0.565	0.968	0.641	1.109	0.722	1.054	0.697	0.764	0.578	0.727	0.588	0.549	0.513	**0.528**	**0.502**		
ETTm1	10	0.788	0.594	0.889	0.630	0.783	0.594	0.706	0.570	0.867	0.660	0.819	0.639	0.723	0.568	0.649	0.566	0.515	0.485	**0.409**	**0.412**	0.403	0.407
	25	0.631	0.528	0.606	0.525	0.639	0.532	0.570	0.514	0.791	0.637	0.728	0.603	0.630	0.528	0.614	0.553	0.479	0.461	**0.406**	**0.409**		
ETTm2	10	0.426	0.434	0.690	0.551	0.454	0.449	0.472	0.461	1.103	0.699	0.905	0.606	0.423	0.435	0.673	0.561	0.348	0.399	**0.341**	**0.393**	0.312	0.370
	25	0.420	0.430	0.477	0.467	0.411	0.430	0.484	0.481	1.073	0.690	0.775	0.559	0.423	0.429	0.633	0.545	0.341	0.391	**0.332**	**0.387**		
Exchange	10	0.343	0.417	0.350	0.426	0.348	0.424	0.351	0.428	0.351	0.428	0.353	0.431	0.354	0.430	0.355	0.433	0.341	0.416	**0.328**	**0.399**	0.319	0.387
	25	0.340	0.411	0.347	0.423	0.343	0.417	0.350	0.429	0.351	0.428	0.351	0.427	0.341	0.414	0.356	0.435	0.331	0.405	**0.325**	**0.395**		
Weather	10	0.277	0.326	0.277	0.334	0.278	0.328	0.283	0.337	0.320	0.394	0.324	0.399	0.280	0.328	0.337	0.409	0.277	0.333	**0.267**	**0.322**	0.265	0.315
	25	0.273	0.323	0.280	0.341	0.273	0.324	0.290	0.358	0.328	0.398	0.332	0.406	0.276	0.327	0.329	0.398	0.272	0.329	**0.265**	**0.318**		
Electricity	10	0.242	0.330	0.269	0.361	0.252	0.343	0.386	0.460	0.678	0.659	0.420	0.486	0.244	0.336	0.494	0.555	0.244	0.337	**0.214**	**0.304**	0.210	0.296
	25	0.227	0.315	0.232	0.323	0.233	0.324	0.335	0.422	0.672	0.656	0.409	0.477	0.230	0.323	0.490	0.551	0.241	0.334	**0.214**	**0.303**		
Traffic	10	0.840	0.496	0.887	0.531	0.874	0.527	1.188	0.707	1.231	0.692	1.173	0.692	0.838	0.498	0.994	0.571	0.727	0.451	**0.625**	**0.384**	0.626	0.385
	25	0.747	0.458	0.751	0.462	0.761	0.471	0.851	0.537	1.173	0.710	1.298	0.721	0.749	0.462	0.966	0.603	0.703	0.441	**0.624**	**0.384**		
1st Count		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	**16**	**16**	–	

configurations provided by their official forecasting implementations.[2] DLinear [46] uses its shared-channel configuration, while iTransformer [24] uses eight attention heads and a dropout rate of 0.1, with its hidden dimensions and encoder depths set according to the dataset-specific scripts. PatchTST [29] follows the supervised forecasting configuration provided for each dataset, using a patch length of 16 and a stride of 8.

4.1.5 Evaluation Platform. All experiments are conducted on a machine with two AMD EPYC 7302 16-Core processors, 256 GB RAM, and an NVIDIA Tesla T4 GPU with 16 GB memory.

4.2 Overall Performance

We first evaluate all condensed datasets using DLinear students. For model-dependent baselines, DLinear also serves as the condensation backbone, yielding a same-architecture comparison. PTC uses DLinear only for downstream evaluation, while its coreset construction relies on the projection model introduced in Section 3.2 and is independent of the student architecture. The full-data result is included as a reference. Table 2 reports the test MSE and MAE averaged over four prediction horizons and five student seeds, with Full Data included as a reference. PTC achieves the lowest MSE and MAE among the compared condensation methods and random selection in all settings. Under the more restrictive budget of $K = 10$, it reduces the MSE over the strongest competing method by approximately 20.6% on ETTm1. Consistent advantages are also observed on the larger Electricity and Traffic datasets, while PTC remains the best method on Exchange and Weather despite the more closely clustered results. With only ten historical windows, PTC also achieves an MSE within 3% of the Full Data reference on five of the eight datasets. This indicates that the constructed targets and retained histories can support forecasting performance close to full-data training under a highly restrictive budget.

[2]https://github.com/cure-lab/LTSF-Linear, https://github.com/thuml/iTransformer, and https://github.com/yuqinie98/PatchTST provide detailed model configurations.

Increasing the budget from 10 to 25 improves forecasting performance in many settings, but the benefit is not uniform across methods. KIP, MTT, and DATM occasionally deteriorate as the budget increases. For PTC, the additional windows bring a more noticeable improvement on ETTh1, while changes on the other datasets are comparatively small. Performance remains nearly unchanged on Electricity and Traffic, and deteriorates slightly on ETTh2. These results suggest that ten retained windows already capture useful forecasting information across the evaluated datasets, while the benefit of retaining more windows varies by dataset. Replacing MTT with DC consistently improves CondTSF, confirming that its performance is sensitive to the iterative condensation backbone, but CondTSF† still remains behind PTC throughout the comparison. Several iterative methods also perform worse than random selection on ETTm2, Electricity, and Traffic. These results show that a larger condensation budget or iterative optimization of synthetic histories does not by itself guarantee better forecasting performance, whereas preserving observed histories and projecting their targets yields consistently strong results across both budgets.

4.3 Cross-Architecture Transferability

We next evaluate cross-architecture transfer by training iTransformer and PatchTST students on the same condensed datasets. Model-dependent baselines use DLinear during condensation, whereas PTC construction is independent of the target student. From Tables 3 and 4, we can observe that the advantage established with DLinear largely persists after changing the student architecture, with PTC achieving the best MSE in 28 of the 32 settings and the best MAE in 31 settings. For example, on ETTh1 with $K = 10$, PTC reduces MSE from 0.607 to 0.511 for iTransformer and from 0.559 to 0.494 for PatchTST relative to the strongest competing baseline for each student. These results indicate that the forecasting information captured by PTC remains useful across student architectures despite occasional ranking changes, without reconstructing the condensed dataset for each model.

Table 3: Cross-architecture transfer performance with iTransformer students. Results are averaged over four prediction horizons $\{96, 192, 336, 720\}$ **and five student seeds.**

Dataset	K	Rand		DC		DM		KIP		MTT		DATM		FRePo		CondTSF		CondTSF†		PTC		Full Data	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	10	0.648	0.549	0.641	0.548	0.671	0.562	0.676	0.562	0.828	0.618	0.809	0.611	0.634	0.543	0.808	0.611	0.607	0.534	**0.511**	**0.482**	0.457	0.449
	25	0.596	0.527	0.591	0.526	0.597	0.529	0.619	0.539	0.829	0.622	0.799	0.608	0.609	0.534	0.799	0.606	0.558	0.512	**0.489**	**0.467**		
ETTh2	10	0.463	0.455	0.439	0.443	0.469	0.458	0.455	0.451	0.465	0.458	0.456	0.453	0.462	0.455	0.464	0.458	0.429	0.438	**0.418**	**0.429**	0.384	0.407
	25	0.448	0.447	0.432	0.439	0.447	0.447	0.432	0.440	0.465	0.458	0.456	0.453	0.449	0.448	0.465	0.458	0.418	0.432	**0.404**	**0.420**		
ETTm1	10	0.620	0.520	0.581	0.502	0.621	0.519	0.664	0.538	0.847	0.602	0.838	0.600	0.606	0.514	0.839	0.599	0.571	0.495	**0.456**	**0.433**	0.416	0.416
	25	0.556	0.489	0.523	0.472	0.556	0.489	0.591	0.507	0.842	0.599	0.820	0.595	0.543	0.483	0.836	0.597	0.518	0.468	**0.446**	**0.429**		
ETTm2	10	0.339	0.370	0.326	0.363	0.342	0.372	0.331	0.366	0.345	0.378	0.341	0.375	0.339	0.369	0.347	0.379	0.318	0.357	**0.311**	**0.352**	0.292	0.336
	25	0.334	0.366	0.319	0.357	0.340	0.372	0.326	0.363	0.344	0.377	0.340	0.374	0.334	0.366	0.346	0.379	0.313	0.353	**0.305**	**0.347**		
Exchange	10	0.432	0.443	0.362	0.414	0.474	0.458	0.406	0.436	0.393	0.429	0.394	0.431	0.444	0.446	0.393	0.429	**0.360**	**0.412**	0.385	0.419	0.379	0.415
	25	0.427	0.435	**0.383**	0.420	0.391	0.425	0.436	0.439	0.393	0.428	0.395	0.432	0.448	0.443	0.393	0.429	0.387	0.421	0.397	**0.419**		
Weather	10	0.304	0.316	0.302	0.316	0.304	0.317	0.300	0.313	0.317	0.330	0.321	0.335	0.306	0.319	0.316	0.329	0.301	0.315	**0.288**	**0.304**	0.259	0.280
	25	0.294	0.307	0.297	0.310	0.294	0.308	0.294	0.310	0.317	0.330	0.320	0.334	0.295	0.309	0.316	0.329	0.296	0.309	**0.285**	**0.301**		
Electricity	10	0.284	0.363	0.319	0.396	0.296	0.376	0.484	0.510	0.691	0.656	0.706	0.634	0.279	0.361	0.506	0.549	0.287	0.370	**0.232**	**0.314**	0.191	0.280
	25	0.259	0.341	0.267	0.351	0.267	0.351	0.427	0.479	0.681	0.650	0.688	0.621	0.258	0.343	0.507	0.546	0.258	0.343	**0.225**	**0.306**		
Traffic	10	0.900	0.523	0.984	0.566	0.943	0.551	1.282	0.712	1.388	0.789	1.444	0.790	0.895	0.521	1.085	0.656	0.775	0.481	**0.663**	**0.411**	0.423	0.282
	25	0.743	0.453	0.744	0.458	0.752	0.463	1.052	0.633	1.344	0.773	1.479	0.807	0.738	0.450	1.103	0.658	0.696	0.439	**0.633**	**0.388**		
1st Count		0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	**14**	**15**	–	

Table 4: Cross-architecture transfer performance with PatchTST students. Results are averaged over four prediction horizons $\{96, 192, 336, 720\}$ **and five student seeds.**

Dataset	K	Rand		DC		DM		KIP		MTT		DATM		FRePo		CondTSF		CondTSF†		PTC		Full Data	
		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	10	0.590	0.518	0.585	0.517	0.599	0.524	0.616	0.532	0.773	0.599	0.743	0.585	0.588	0.518	0.751	0.588	0.559	0.508	**0.494**	**0.471**	0.452	0.441
	25	0.534	0.492	0.527	0.487	0.536	0.493	0.556	0.502	0.774	0.599	0.741	0.584	0.541	0.497	0.751	0.588	0.523	0.491	**0.476**	**0.460**		
ETTh2	10	0.443	0.443	0.429	0.430	0.443	0.443	0.436	0.439	0.451	0.446	0.446	0.444	0.441	0.441	0.453	0.447	0.411	0.424	**0.400**	**0.418**	0.378	0.402
	25	0.427	0.432	0.410	0.422	0.426	0.432	0.410	0.424	0.453	0.447	0.446	0.443	0.427	0.432	0.453	0.448	0.398	0.417	**0.390**	**0.411**		
ETTm1	10	0.564	0.492	0.524	0.474	0.570	0.494	0.600	0.509	0.747	0.567	0.702	0.549	0.558	0.489	0.741	0.562	0.510	0.466	**0.436**	**0.420**	0.380	0.394
	25	0.508	0.464	0.483	0.449	0.511	0.466	0.514	0.468	0.748	0.568	0.682	0.541	0.505	0.463	0.734	0.559	0.479	0.449	**0.426**	**0.414**		
ETTm2	10	0.326	0.360	0.309	0.349	0.327	0.362	0.318	0.356	0.325	0.361	0.318	0.356	0.325	0.359	0.323	0.360	0.308	0.348	**0.302**	**0.343**	0.286	0.328
	25	0.321	0.356	0.307	0.347	0.326	0.361	0.315	0.354	0.324	0.361	0.316	0.354	0.322	0.357	0.322	0.360	0.307	0.347	**0.300**	**0.341**		
Exchange	10	0.401	0.424	0.385	0.420	0.412	0.430	0.397	0.430	0.391	0.426	0.394	0.430	0.410	0.429	0.389	0.425	0.388	0.423	**0.380**	**0.413**	0.350	0.396
	25	**0.372**	0.410	0.373	0.410	0.378	0.415	0.390	0.423	0.391	0.426	0.394	0.430	0.381	0.417	0.389	0.425	0.384	0.420	0.376	**0.409**		
Weather	10	0.297	0.311	0.295	0.311	0.295	0.311	0.289	0.307	0.298	0.321	0.301	0.322	0.299	0.314	0.295	0.317	0.288	0.307	**0.283**	**0.299**	0.257	0.280
	25	0.289	0.303	0.288	0.302	0.287	0.302	0.283	0.304	0.298	0.321	0.300	0.321	0.290	0.305	0.295	0.317	**0.278**	0.298	0.280	**0.297**		
Electricity	10	0.251	0.332	0.279	0.362	0.261	0.344	0.468	0.508	0.684	0.652	0.699	0.631	0.251	0.336	0.513	0.549	0.262	0.349	**0.221**	**0.303**	0.191	0.276
	25	0.232	0.314	0.237	0.320	0.239	0.324	0.393	0.461	0.683	0.651	0.695	0.625	0.235	0.322	0.519	0.550	0.251	0.337	**0.220**	**0.300**		
Traffic	10	0.795	0.468	0.835	0.500	0.817	0.488	1.032	0.627	1.383	0.788	1.340	0.764	0.794	0.471	1.191	0.697	0.745	0.461	**0.637**	**0.392**	0.466	0.292
	25	0.717	0.446	0.719	0.449	0.727	0.454	0.811	0.515	1.388	0.787	1.291	0.756	0.721	0.449	1.214	0.705	0.720	0.447	**0.634**	**0.390**		
1st Count		1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	**14**	**16**	–	

Despite using the same target-correction strategy, the two variants of CondTSF differ substantially in cross-architecture performance. Replacing MTT with DC consistently improves performance for both students, indicating that target correction alone does not eliminate the influence of the underlying condensation method. MTT and DATM also frequently underperform random selection, showing that iterative condensation does not consistently produce training data that support different student architectures. By comparison, PTC retains its overall advantage without matching downstream gradients or training trajectories. This transferability, however, does not imply equivalence to full-data training. On Traffic with $K = 25$, its MSE remains approximately 0.63 across all three students, whereas full-data iTransformer and PatchTST achieve 0.423 and 0.466, respectively. The stronger forecasting architectures therefore bring limited improvement when trained on the PTC datasets in this setting, despite their better performance with the full data. These results distinguish the ability of a condensed dataset to support multiple student architectures from its ability to preserve the information needed to exploit their full predictive capacity.

4.4 Ablation Study

Target construction. The PTC framework constructs targets using a generator fitted on the complete training dataset. We use the closed-form linear projector introduced in Section 3.2 by default and examine DLinear, iTransformer, and PatchTST as alternative target generators while keeping the randomly selected historical windows identical. As shown in Table 5, retaining the observed futures does not achieve a single best result across the eight datasets under either condensation budget for any of the three students. The best target generator instead follows a dataset-specific pattern. PatchTST-generated targets are generally the strongest on ETTh1 across all three students, whereas PTC targets are generally the strongest on Traffic. A target generator is not consistently better when it

Table 5: Target-generator ablation with fixed random inputs. All variants use the same input windows and differ only in target construction. Results are averaged over four prediction horizons {96, 192, 336, 720} and five student seeds.

Student	Target generator	K	ETTh1 MSE	ETTh1 MAE	ETTh2 MSE	ETTh2 MAE	ETTm1 MSE	ETTm1 MAE	ETTm2 MSE	ETTm2 MAE	Exchange MSE	Exchange MAE	Weather MSE	Weather MAE	Electricity MSE	Electricity MAE	Traffic MSE	Traffic MAE	1st Count MSE	1st Count MAE
DLinear	True Y	10	0.843	0.633	0.779	0.587	0.788	0.594	0.426	0.434	0.343	0.417	0.277	0.326	0.242	0.330	0.840	0.496	0	0
		25	0.674	0.572	0.687	0.553	0.631	0.528	0.420	0.430	0.340	0.411	0.273	0.323	0.227	0.315	0.748	0.458		
	PTC Y	10	0.496	0.477	0.528	0.502	0.418	0.419	0.334	0.389	0.341	0.413	**0.269**	0.322	**0.214**	0.304	**0.624**	**0.384**	7	3
		25	0.467	0.461	0.524	0.500	**0.407**	**0.411**	0.334	0.388	0.326	0.397	**0.266**	0.317	**0.214**	0.304	**0.624**	**0.384**		
	DLinear Y	10	0.493	0.480	0.487	0.479	**0.416**	**0.417**	**0.312**	0.371	0.327	0.396	0.269	0.323	0.215	0.304	0.626	0.385	2	1
		25	0.492	0.483	0.481	0.477	0.408	0.412	0.309	0.368	0.324	0.393	0.266	0.318	0.214	0.303	0.626	0.385		
	iTransformer Y	10	0.497	0.473	0.416	0.425	0.550	0.493	0.312	0.358	**0.315**	**0.385**	0.278	0.311	0.228	0.309	0.763	0.455	2	2
		25	0.475	0.459	0.399	0.417	0.520	0.477	0.314	0.354	**0.319**	**0.385**	0.271	0.307	0.223	0.305	0.705	0.431		
	PatchTST Y	10	**0.462**	**0.450**	**0.392**	**0.411**	0.477	0.449	0.312	**0.355**	0.450	0.444	0.278	**0.308**	0.222	**0.304**	0.758	0.448	5	10
		25	**0.454**	**0.444**	**0.382**	**0.405**	0.448	0.433	**0.308**	**0.352**	0.350	0.397	0.272	**0.303**	0.219	**0.301**	0.697	0.422		
iTransformer	True Y	10	0.647	0.549	0.463	0.455	0.621	0.520	0.339	0.370	0.427	0.441	0.304	0.316	0.284	0.363	0.902	0.524	0	0
		25	0.595	0.526	0.447	0.447	0.556	0.489	0.334	0.366	0.416	0.432	0.294	0.308	0.259	0.341	0.742	0.453		
	PTC Y	10	0.505	0.478	0.418	0.428	**0.465**	**0.441**	0.311	0.350	0.384	0.420	**0.294**	0.307	0.229	0.311	**0.671**	**0.418**	4	3
		25	0.491	0.469	0.398	0.417	**0.439**	**0.424**	0.305	0.345	0.421	0.431	0.291	0.304	0.225	0.306	0.638	0.392		
	DLinear Y	10	0.519	0.487	0.410	0.423	0.467	0.442	0.306	0.346	0.382	0.417	0.294	0.307	**0.229**	**0.311**	0.673	0.420	1	1
		25	0.508	0.480	0.395	0.415	0.440	0.425	0.301	0.342	0.410	0.426	0.291	0.305	0.225	0.306	0.639	0.393		
	iTransformer Y	10	0.502	0.478	0.399	0.416	0.556	0.487	**0.305**	**0.345**	0.366	0.410	0.298	0.309	0.239	0.321	0.760	0.460	2	2
		25	0.477	0.462	0.386	0.409	0.516	0.468	0.301	0.341	**0.345**	**0.400**	0.290	0.302	0.226	0.310	0.645	0.399		
	PatchTST Y	10	**0.485**	**0.467**	**0.393**	**0.413**	0.482	0.450	0.306	0.345	**0.359**	**0.404**	0.297	**0.306**	0.231	0.313	0.732	0.448	9	10
		25	**0.466**	**0.454**	**0.385**	**0.409**	0.448	0.430	**0.300**	**0.341**	0.360	0.404	**0.285**	**0.297**	**0.221**	**0.304**	**0.612**	**0.384**		
PatchTST	True Y	10	0.590	0.519	0.442	0.441	0.566	0.492	0.327	0.361	0.397	0.421	0.298	0.312	0.252	0.332	0.796	0.468	0	0
		25	0.534	0.492	0.425	0.430	0.507	0.464	0.321	0.357	0.369	0.408	0.290	0.303	0.233	0.314	0.719	0.447		
	PTC Y	10	0.493	0.470	0.402	0.416	**0.436**	**0.422**	0.299	0.340	0.379	0.411	0.278	0.296	**0.222**	0.302	**0.635**	**0.392**	6	5
		25	**0.471**	**0.456**	0.387	0.407	**0.419**	**0.409**	0.295	0.336	0.375	0.407	0.275	0.294	0.220	0.300	**0.636**	**0.393**		
	DLinear Y	10	0.498	0.473	0.398	0.414	0.437	0.423	0.297	0.337	0.379	0.411	0.278	0.297	0.222	0.303	0.635	0.393	0	0
		25	0.478	0.460	0.387	0.407	0.419	0.410	0.292	0.333	0.374	0.408	0.275	0.294	0.221	0.301	0.637	0.394		
	iTransformer Y	10	0.502	0.476	0.397	0.414	0.526	0.474	0.303	0.344	**0.378**	**0.409**	0.286	0.299	0.228	0.308	0.736	0.439	2	2
		25	0.481	0.463	0.389	0.408	0.491	0.457	0.300	0.340	**0.366**	**0.404**	0.274	0.291	0.222	0.303	0.690	0.429		
	PatchTST Y	10	**0.491**	**0.469**	**0.388**	**0.408**	0.455	0.435	**0.294**	**0.335**	0.380	0.413	**0.277**	**0.292**	0.222	**0.302**	0.717	0.429	8	9
		25	0.472	0.457	**0.383**	**0.405**	0.424	0.415	**0.288**	**0.330**	0.379	0.411	**0.270**	**0.288**	**0.218**	**0.299**	0.675	0.421		

shares the student's architecture. Although PatchTST-generated targets obtain the largest number of best results overall, no target generator dominates every dataset. The default PTC targets remain competitive across all three students and require no separately trained neural target generator, making the closed-form projection an efficient default rather than the only possible target-construction mechanism.

We further examine whether the benefit of target projection depends on the real historical windows retained by PTC or can also transfer to synthetically generated inputs. For this experiment, we use the inputs produced by CondTSF†, the best-performing baseline in the overall comparison, and replace only its targets with PTC targets. As shown in Figure 3, target replacement improves forecasting performance in most evaluated settings. The gains are particularly evident on ETTh1, ETTm1, Electricity, and Traffic, and tend to be larger under the more restrictive budget of $K = 10$. Exchange is the only dataset on which the effect is not consistent across settings. Since neither the synthetic inputs nor the student-training procedure is changed, the improvements can be attributed directly to the constructed targets. These results show that target projection is not merely coupled with historical-window selection in PTC, but can also strengthen a competitive iterative condensation method.

Historical-window selection. We next examine the additional contribution of leverage-based historical-window selection, with targets for both selection methods generated by the same PTC projector. Figure 4 compares the selected windows with randomly sampled windows for the DLinear student. Each point reports the

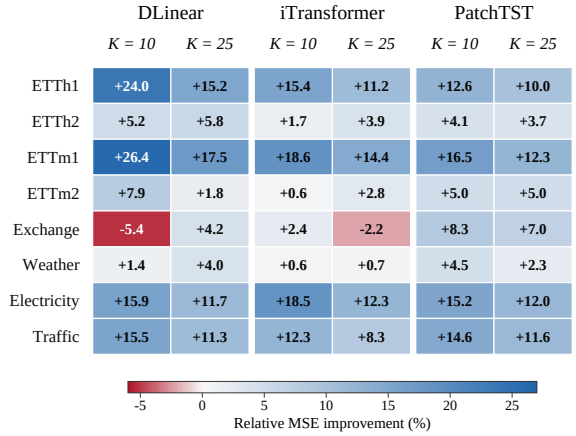


Figure 3: Effect of replacing CondTSF† targets with PTC targets while keeping its synthetic inputs fixed. Cells report relative MSE improvements averaged over four prediction horizons and five student seeds.

relative MSE improvement over random selection for a fixed prediction horizon and condensation budget, computed from MSE averaged over five student initializations. Projection-aware selection improves MSE in 44 of the 64 evaluated configurations, with an average relative improvement of approximately 0.52%. The additional benefit is most consistent on Electricity, Weather, and ETTm1, while results on the remaining datasets are more variable. On Exchange, for example, selection yields an average MSE reduction of

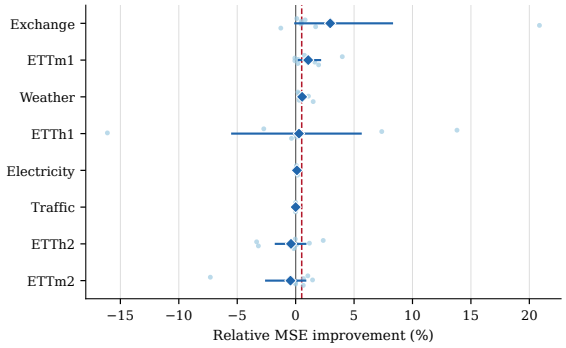


Figure 4: Dataset-wise effect of leverage-based historical-window selection under projected targets for DLinear students. Points show relative MSE improvements over random selection. Diamonds and horizontal lines denote dataset means and 95% confidence intervals.

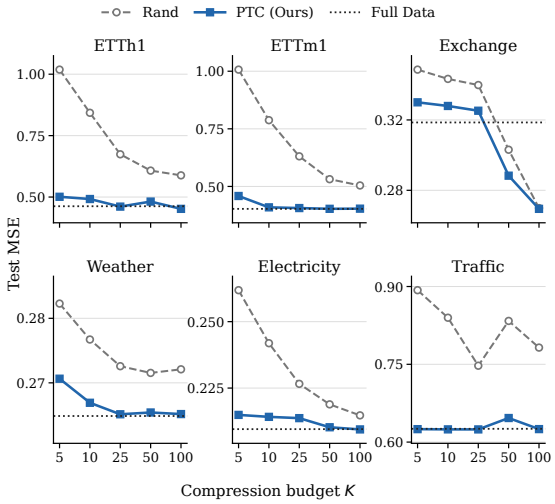


Figure 5: Test MSE under different compression budgets for DLinear students. Results are averaged over four prediction horizons and five student seeds.

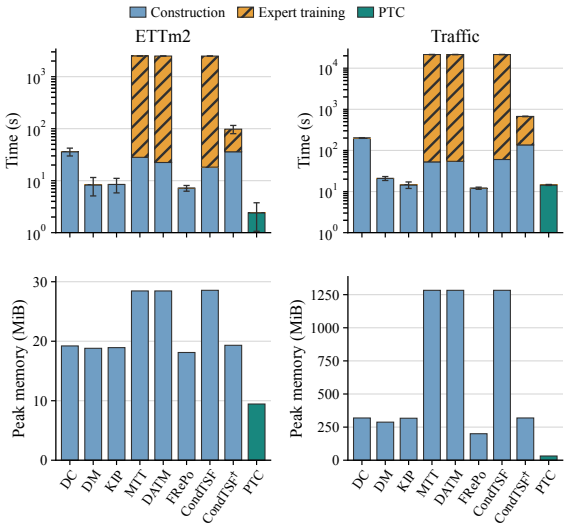


Figure 6: Dataset-construction time and peak construction memory on ETTm2 and Traffic. Expert-training time is stacked for methods that require pretrained experts.

approximately 3%, with gains exceeding 20% in an individual setting. These results identify target construction as the primary source of improvement, while leverage-based historical-window selection provides a smaller and more dataset-dependent refinement.

4.5 Sensitivity Study

We further evaluate the sensitivity to the compression budget by extending K to $\{5, 10, 25, 50, 100\}$ on six datasets with DLinear students, corresponding to approximately 0.014%–1.95% of the original training samples. Figure 5 shows how PTC performance changes with the condensation budget, with random selection included as a reference. Even at $K = 5$, PTC already approaches the Full Data reference on most datasets. The benefit of increasing K varies across datasets and budget ranges. On Weather, most of the improvement occurs by $K = 25$, whereas Exchange continues to benefit substantially from larger budgets, with both PTC and random selection eventually surpassing the Full Data reference. We conjecture that a moderately sized subset provides sufficient coverage of the temporal patterns in Exchange while reducing the repeated emphasis on redundant overlapping windows, which may have a regularizing effect on student training. As a reference, random selection generally benefits from additional windows, while PTC consistently outperforms it across the evaluated budgets, with the largest differences appearing at lower budgets. Appendix B.1 extends the budget sensitivity study to iTransformer students and alternative target generators.

4.6 Efficiency

We evaluate efficiency from both dataset construction and downstream student training. Figure 6 compares construction time and peak GPU memory on ETTm2 and Traffic with $H = 96$ and $K = 25$ using a Tesla T4 GPU. Construction time is averaged over five runs, while peak memory covers the construction stage only. For methods requiring pretrained experts, the stacked bars additionally include expert preparation. MTT, DATM, and the MTT-based CondTSF use a 40-model expert pool, whereas the DC-based CondTSF† requires one expert. PTC constructs the datasets in approximately 2.4 seconds on ETTm2 and 14.5 seconds on Traffic. It is the fastest method on ETTm2 and remains comparable to the fastest baselines on Traffic, while using the least memory on both datasets. The difference becomes substantially larger when expert preparation is included, showing that the closed-form construction avoids the principal computational overhead of trajectory-based condensation.

Figure 7 further examines the cost of training models on the resulting datasets. We compare Full Data with PTC at $K = 25$ on ETTm2 and Traffic using DLinear, iTransformer, and PatchTST students. The curves report the median and interquartile range over five seeds, and validation overhead is excluded from cumulative training time. Across both datasets, PTC reaches a stable validation error substantially earlier than Full Data. On ETTm2, this occurs within a few seconds for all three students, with a particularly large training-time difference for PatchTST. The same efficiency

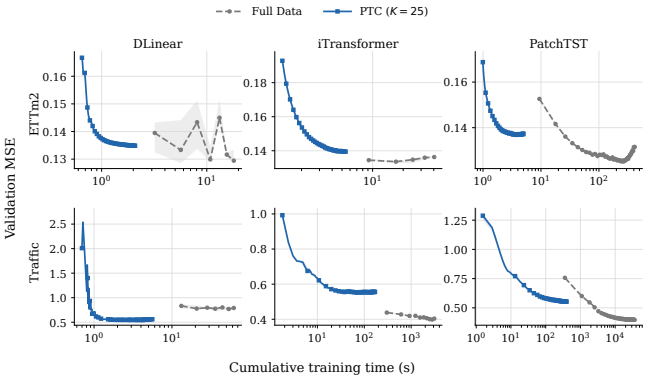


Figure 7: Validation MSE against cumulative student-training time on ETTm2 and Traffic for Full Data and PTC with three student architectures.

advantage holds on Traffic, although iTransformer and PatchTST require longer training than on ETTm2. In addition, Full Data attains lower final errors on ETTm2 and for the two Transformer students on Traffic, whereas PTC achieves both lower validation error and shorter training time for DLinear on Traffic. In the latter setting, construction and training together take approximately 23 seconds, compared with 61 seconds for full-data training, recovering the construction cost before any further reuse of the condensed dataset. By contrast, the evaluated trajectory-based baselines, including MTT, DATM, and MTT-based CondTSF, incur expert-preparation costs alone that far exceed the cost of a single full-data DLinear training run. Appendix B.2 extends the evaluation to LargeST-CA, a substantially larger dataset, examining construction cost, student-training time, and forecasting accuracy.

5 RELATED WORK

5.1 Coreset Selection and Dataset Condensation

Coreset selection [14, 27, 38, 45] reduces the size of a training set by retaining representative real samples, commonly according to geometric coverage [34], sensitivity [3], or statistical importance [5, 8]. Since the selected samples preserve their original inputs and targets, the resulting subset remains directly interpretable, but its quality is constrained by the information contained in individual observations [47]. Dataset condensation adopts a task-aware alternative that constructs a compact training set to reproduce the learning effect of the original data [12, 23, 25]. Gradient and distribution matching align local training signals or feature statistics between the synthetic and original datasets [48, 49], while kernel and neural feature methods optimize the synthetic data through surrogate prediction objectives [28, 52]. Trajectory-based methods (e.g., MTT) further match the parameter evolution induced by the two datasets [4, 13]. Despite their different objectives, these methods generally treat synthetic inputs as the primary carrier of compressed information and obtain them through iterative optimization. A complementary line of research studies soft-label construction and label distillation [2, 32, 37], showing that supervision can encode substantial knowledge from the original dataset even when the inputs are fixed. These studies, however, mainly concern classification, where the constructed targets represent class identities or probability distributions rather than continuous future trajectories.

5.2 Dataset Condensation for Time-Series Forecasting

Recent studies have extended dataset condensation to time-series forecasting [7, 19, 26, 30, 36]. TimeDC preserves temporal information through frequency-domain matching and curriculum-based trajectory matching, but still requires synthetic sequences to be optimized against stored expert trajectories [26]. CondTSF jointly optimizes synthetic historical inputs and future targets through an underlying condensation method, with periodic target correction using predictions from a model trained on the full dataset [7]. Although this correction explicitly adjusts the supervision, it remains coupled to the iterative optimization of synthetic historical inputs. In addition, broader temporal structure is incorporated in UniTSC through alignment of optimization trajectories, spectral properties, and multivariate dependencies [36]. For spatio-temporal forecasting, STemDist compresses both temporal and spatial dimensions, combining cluster-level distillation with subset-based granular distillation [19]. In contrast, our work highlights target construction as a central consideration in time-series dataset condensation, alongside the selection or synthesis of historical inputs. When only a few training windows are retained, the residual variation in their observed futures can obscure the relationship between history and future. This motivates pairing real historical windows with targets estimated from the complete training set. The linear projection enables closed-form construction without iterative input synthesis.

6 CONCLUSION AND FUTURE WORK

This paper presents Projected-Target Coresets (PTC), a framework for efficient time-series dataset condensation. The method is motivated by decomposing each realized future into a component predictable from its historical window and a sample-specific residual. Although residual variation can average out during full-data training, it may dominate the learning signal when only a few observed futures are retained. PTC therefore preserves real historical windows while replacing their observed targets with predictable components estimated from the complete training set. The default linear implementation combines target projection and leverage-based historical-window selection in a shared representation, enabling closed-form construction without iterative input synthesis. Experiments across eight forecasting benchmarks demonstrate consistent improvements over random selection and existing condensation methods under different budgets and student architectures, while the efficiency evaluation confirms substantially lower construction cost. The ablation results further identify target projection as the main source of improvement, with projection-aware selection providing an additional but more dataset-dependent benefit.

Future work may develop adaptive target decompositions and diversity-aware selection strategies to better represent nonlinear dynamics and distinct temporal regimes, particularly on challenging datasets such as Traffic. Extending the framework to nonstationary data streams and probabilistic forecasting would also enable target uncertainty to be modeled explicitly rather than removed as residual variation.

REFERENCES

[1] Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. 2023. Accurate medium-range global weather forecasting with 3D neural networks. *Nature* 619, 7970 (2023), 533–538.

[2] Ondrej Bohdal, Yongxin Yang, and Timothy Hospedales. 2020. Flexible dataset distillation: Learn labels instead of images. *arXiv preprint arXiv:2006.08572* (2020).

[3] Vladimir Braverman, Dan Feldman, Harry Lang, Adiel Statman, and Samson Zhou. 2021. Efficient Coreset Constructions via Sensitivity Sampling. In *ACML*, Vol. 157. 948–963.

[4] George Cazenavette, Tongzhou Wang, Antonio Torralba, Alexei A. Efros, and Jun-Yan Zhu. 2022. Dataset Distillation by Matching Training Trajectories. In *CVPR Workshop*. 4749–4758.

[5] Michael B Cohen, Cameron Musco, and Christopher Musco. 2017. Input sparsity time low-rank approximation via ridge leverage score sampling. In *SODA*. 1758–1777.

[6] Liwei Deng, Tianfu Wang, Yan Zhao, and Kai Zheng. 2024. MILLION: A General Multi-Objective Framework with Controllable Risk for Portfolio Management. *PVLDB* 18, 2 (2024), 466–474.

[7] Jianrong Ding, Zhanyu Liu, Guanjie Zheng, Haiming Jin, and Linghe Kong. 2024. CondTSF: One-line Plugin of Dataset Condensation for Time Series Forecasting. In *NeurIPS*, Vol. 37. 128227–128259.

[8] Petros Drineas, Malik Magdon-Ismail, Michael W Mahoney, and David P Woodruff. 2012. Fast approximation of matrix coherence and statistical leverage. *JMLR* 13, 1 (2012), 3475–3506.

[9] Yuchen Fang, Yuxuan Liang, Bo Hui, Zezhi Shao, Liwei Deng, Xu Liu, Xinke Jiang, and Kai Zheng. 2025. Efficient large-scale traffic forecasting with transformers: A spatial data management perspective. In *KDD*. 307–317.

[10] Yuchen Fang, Hao Miao, Yuxuan Liang, Liwei Deng, Yue Cui, Ximu Zeng, Yuyang Xia, Yan Zhao, Torben Bach Pedersen, Christian S Jensen, et al. 2026. Unraveling spatio-temporal foundation models via the pipeline lens: A comprehensive review. *TKDE* (2026).

[11] Yuchen Fang, Yanjun Qin, Haiyong Luo, Fang Zhao, Bingbing Xu, Liang Zeng, and Chenxing Wang. 2023. When spatio-temporal meet wavelets: Disentangled traffic forecasting via efficient spectral graph attention networks. In *ICDE*. IEEE, 517–529.

[12] Xinyi Gao, Tong Chen, Yilong Zang, Wentao Zhang, Quoc Viet Hung Nguyen, Kai Zheng, and Hongzhi Yin. 2024. Graph condensation for inductive node representation learning. In *ICDE*. 3056–3069.

[13] Ziyao Guo, Kai Wang, George Cazenavette, Hui Li, Kaipeng Zhang, and Yang You. 2024. Towards Lossless Dataset Distillation via Difficulty-Aligned Trajectory Matching. In *ICLR*.

[14] Aviv Hadar, Tova Milo, and Kathy Razmadze. 2024. Datamap-driven tabular coreset selection for classifier training. *PVLDB* 18, 3 (2024), 876–888.

[15] Jindong Han, Hao Wang, Hui Xiong, and Hao Liu. 2025. Scalable Pre-Training of Compact Urban Spatio-Temporal Predictive Models on Large-Scale Multi-Domain Data. *PVLDB* 18, 7 (2025), 2149–2158.

[16] Jindong Han, Weijia Zhang, Hao Liu, Tao Tao, Naiqiang Tan, and Hui Xiong. 2024. Bigst: Linear complexity spatio-temporal graph neural network for traffic forecasting on large-scale road networks. *PVLDB* 17, 5 (2024), 1081–1090.

[17] Yujia Huang, Qiuye Sun, Zhe Chen, David Wenzhong Gao, Torben Bach Pedersen, Kim Guldstrand Larsen, and Yushuai Li. 2025. Dynamic modeling and analysis for electricity-gas systems with electric-driven compressors. *TSG* 16, 3 (2025), 2144–2155.

[18] Weicong Kong, Zhao Yang Dong, Youwei Jia, David J Hill, Yan Xu, and Yuan Zhang. 2017. Short-term residential load forecasting based on LSTM recurrent neural network. *TSG* 10, 1 (2017), 841–851.

[19] Taehyung Kwon, Yeonje Choi, Yeongho Kim, and Kijung Shin. 2026. Effective Dataset Distillation for Spatio-Temporal Forecasting with BI-Dimensional Compression. In *ICDE*. 1–14.

[20] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. 2018. Modeling long-and short-term temporal patterns with deep neural networks. In *SIGIR*. 95–104.

[21] Zhichen Lai, Dalin Zhang, Huan Li, Christian S Jensen, Hua Lu, and Yan Zhao. 2023. Lightcts: A lightweight framework for correlated time series forecasting. *SIGMOD* 1, 2 (2023), 1–26.

[22] Remi Lam, Alvaro Sanchez-Gonzalez, Matthew Willson, Peter Wirnsberger, Meire Fortunato, Ferran Alet, Suman Ravuri, Timo Ewalds, Zach Eaton-Rosen, Weihua Hu, et al. 2023. Learning skillful medium-range global weather forecasting. *Science* 382, 6677 (2023), 1416–1421.

[23] Yuxuan Liang, Wentao Zhang, Xinyi Gao, Ling Yang, Chong Chen, Hongzhi Yin, Yunhai Tong, and Bin Cui. 2025. Training-free heterogeneous graph condensation via data selection. In *ICDE*. 1718–1731.

[24] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. itransformer: Inverted transformers are effective for time series forecasting. In *ICLR*, Vol. 2024. 11116–11140.

[25] Zhanyu Liu, Ke Hao, Guanjie Zheng, and Yanwei Yu. 2024. Dataset condensation for time series classification via dual domain matching. In *KDD*. 1980–1991.

[26] Hao Miao, Ziqiao Liu, Yan Zhao, Chenjuan Guo, Bin Yang, Kai Zheng, and Christian S Jensen. 2024. Less is More: Efficient Time Series Dataset Condensation via Two-fold Modal Matching. *PVLDB* 18, 2 (2024), 226–238.

[27] Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. 2020. Coresets for data-efficient training of machine learning models. In *ICML*. 6950–6960.

[28] Timothy Nguyen, Zhourong Chen, and Jaehoon Lee. 2021. Dataset Meta-Learning from Kernel-Ridge Regression. In *ICLR*.

[29] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *ICLR*.

[30] Yiming Niu, Jinliang Deng, Lulu Zhang, Zimu Zhou, and Yongxin Tong. 2025. Accurate and efficient multivariate time series forecasting via offline clustering. In *ICDE*. 3056–3069.

[31] Weizhu Qian, Yan Zhao, Dalin Zhang, Bowei Chen, Kai Zheng, and Xiaofang Zhou. 2023. Towards a unified understanding of uncertainty quantification in traffic flow forecasting. *TKDE* 36, 5 (2023), 2239–2256.

[32] Tian Qin, Zhiwei Deng, and David Alvarez-Melis. 2024. A label is worth a thousand images in dataset distillation. *NeurIPS* 37 (2024), 131946–131971.

[33] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. 2024. TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods. *PVLDB* 17, 9 (2024), 2363–2377.

[34] Ozan Sener and Silvio Savarese. 2018. Active Learning for Convolutional Neural Networks: A Core-Set Approach. In *ICLR*.

[35] Syed Yousaf Shah, Dhaval Patel, Long Vu, Xuan-Hong Dang, Bei Chen, Peter Kirchner, Horst Samulowitz, David Wood, Gregory Bramble, Wesley M Gifford, et al. 2021. AutoAI-TS: AutoAI for time series forecasting. In *SIGMOD*. 2584–2596.

[36] Wei Shao, Ziquan Fang, Zheqi Lu, Yongfeng Su, Yuzhu Wang, and Yunjun Gao. 2026. One Batch Is Enough: A Unified Dataset Condensation Framework for General Time Series Analysis. In *ICML*, Vol. 306. 110102–110130.

[37] Ilia Sucholutsky and Matthias Schonlau. 2021. Soft-label dataset distillation and text dataset distillation. In *IJCNN*. 1–8.

[38] Jiayi Wang, Chengliang Chai, Nan Tang, Jiabin Liu, and Guoliang Li. 2022. Coresets over multiple tables for feature-rich and data-efficient machine learning. *PVLDB* 16, 1 (2022), 64–76.

[39] Kai Wang, Bo Zhao, Xiangyu Peng, Zheng Zhu, Shuo Yang, Shuo Wang, Guan Huang, Hakan Bilen, Xinchao Wang, and Yang You. 2022. Cafe: Learning to condense dataset by aligning features. In *CVPR*. 12186–12195.

[40] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A Efros. 2018. Dataset distillation. *arXiv preprint arXiv:1811.10959* (2018).

[41] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS* 34 (2021), 22419–22430.

[42] Xinle Wu, Xingjian Wu, Dalin Zhang, Miao Zhang, Chenjuan Guo, Bin Yang, and Christian S. Jensen. 2024. Fully Automated Correlated Time Series Forecasting in Minutes. *PVLDB* 18, 2 (2024), 144–157.

[43] Xinle Wu, Dalin Zhang, Chenjuan Guo, Chaoyang He, Bin Yang, and Christian S Jensen. 2021. AutoCTS: Automated Correlated Time Series Forecasting. *PVLDB* 15, 4 (2021), 971–983.

[44] Xinle Wu, Dalin Zhang, Miao Zhang, Chenjuan Guo, Bin Yang, and Christian S Jensen. 2023. AutoCTS+: Joint neural architecture and hyperparameter search for correlated time series forecasting. *SIGMOD* 1, 1 (2023), 1–26.

[45] Haitao Yuan and Gao Cong. 2026. GoodTP: An Effective Data Selection Framework for Enhancing Trajectory Similarity Learning via Monte Carlo Tree Search. *SIGMOD* 4, 3 (2026), 1–27.

[46] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *AAAI*, Vol. 37. 11121–11128.

[47] Bo Zhao and Hakan Bilen. 2021. Dataset condensation with differentiable siamese augmentation. In *ICML*. 12674–12685.

[48] Bo Zhao and Hakan Bilen. 2023. Dataset Condensation with Distribution Matching. In *WACV*. 6514–6523.

[49] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2021. Dataset Condensation with Gradient Matching. In *ICLR*.

[50] Zetao Zheng, Jie Shao, Jia Zhu, and Heng Tao Shen. 2023. Relational temporal graph convolutional networks for ranking-based stock prediction. In *ICDE*. IEEE, 123–136.

[51] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, Vol. 35. 11106–11115.

[52] Yongchao Zhou, Ehsan Nezhadarya, and Jimmy Ba. 2022. Dataset Distillation using Neural Feature Regression. In *NeurIPS*, Vol. 35. 9813–9827.

A ADDITIONAL THEORETICAL ANALYSIS

A.1 Proof of Proposition 3.1

Proof. Differentiating the two losses with respect to θ gives:

$$\begin{aligned}\Delta_{\mathcal{I}} &= -\frac{2}{KCH}\sum_{i\in\mathcal{I}}\mathbf{J}_i(\theta)^\top \operatorname{vec}(\mathbf{R}_i)\\ &= \frac{1}{K}\sum_{i\in\mathcal{I}}\mathbf{d}_i.\end{aligned} \tag{20}$$

For uniform sampling without replacement, let a_i indicate whether window i is included in $\mathcal{I}$. Then $\mathbb{E}[a_i] = K/N$ and $\mathbb{E}[a_i a_j] = K(K-1)/(N(N-1))$ for $i \neq j$, which immediately gives $\mathbb{E}_{\mathcal{I}}[\Delta_{\mathcal{I}}] = \Delta_{\text{full}}$.

To obtain the second moment, write $\mathbf{u}_i = \mathbf{d}_i - \Delta_{\text{full}}$, so that $\sum_i \mathbf{u}_i = \mathbf{0}$ and $\sum_i \|\mathbf{u}_i\|_2^2 = NV_{\mathbf{d}}$. Consequently, $\sum_{i\neq j}\mathbf{u}_i^\top\mathbf{u}_j = -NV_{\mathbf{d}}$. Applying the inclusion probabilities gives the following equation.

$$\begin{aligned}\mathbb{E}_{\mathcal{I}}\left[\|\Delta_{\mathcal{I}} - \Delta_{\text{full}}\|_2^2\right] &= \frac{1}{K^2}\left(\frac{K}{N}NV_{\mathbf{d}} - \frac{K(K-1)}{N(N-1)}NV_{\mathbf{d}}\right)\\ &= \frac{N-K}{K(N-1)}V_{\mathbf{d}}.\end{aligned} \tag{21}$$

□

A.2 Effect of Target Projection on a Linear Student

The gradient analysis in Section 3.4 characterizes how projection residuals contribute to training on retained subsets. We complement this analysis by comparing the expected prediction errors of linear students trained with observed and projected targets.

Conditioned on the historical representation matrix $\mathbf{Z} \in \mathbb{R}^{n\times d}$, where $n = NC$, assume $\mathbf{T} = \mathbf{M} + \mathbf{E}$ with $\mathbf{M} = \mathbb{E}[\mathbf{T} \mid \mathbf{Z}] = \mathbf{Z}\Theta_\star$. Here, $\mathbf{E}$ denotes the deviation from the conditional mean and therefore satisfies $\mathbb{E}[\mathbf{E} \mid \mathbf{Z}] = \mathbf{0}$. Unlike the projection residual $\mathbf{R} = (\mathbf{I}_n - \mathbf{P}_Z)\mathbf{T}$, it is defined relative to the underlying conditional mean rather than the fitted operator. We additionally assume $\mathbb{E}[\mathbf{E}\mathbf{E}^\top \mid \mathbf{Z}] = \tau^2\mathbf{I}_n$, so each channel–window pair has the same total residual variance, and the expected inner product between the residual vectors of any two distinct pairs is zero.

Let $(\cdot)^\dagger$ denote the pseudoinverse and $\mathbf{P}_Z = \mathbf{Z}(\mathbf{Z}^\top\mathbf{Z})^\dagger\mathbf{Z}^\top$. Under the assumed linear conditional mean, the projection used by PTC satisfies $\mathbf{P}_Z\mathbf{M} = \mathbf{P}_Z\mathbf{Z}\Theta_\star = \mathbf{Z}\Theta_\star = \mathbf{M}$, where $\mathbf{P}_Z\mathbf{Z} = \mathbf{Z}$ follows from projection onto the column space of $\mathbf{Z}$. The projected targets consequently satisfy $\widehat{\mathbf{T}} = \mathbf{M} + \mathbf{P}_Z\mathbf{E}$.

Let $\mathcal{I}$ contain K retained window indices, fixed independently of the residual realizations conditional on $\mathbf{Z}$. Define $\mathbf{S}_{\mathcal{I}} \in \mathbb{R}^{KC\times n}$ to select all channel rows associated with these windows, and let $\mathbf{A} = \mathbf{S}_{\mathcal{I}}\mathbf{Z}$. Fitting a channel-independent linear student with parameters shared across channels by minimum-norm least squares gives $\widehat{\Theta}_{\text{obs}} = \mathbf{A}^\dagger\mathbf{S}_{\mathcal{I}}\mathbf{T}$ for observed targets and $\widehat{\Theta}_{\text{proj}} = \mathbf{A}^\dagger\mathbf{S}_{\mathcal{I}}\mathbf{P}_Z\mathbf{T}$ for projected targets. We compare their predictions on fixed test representations $\mathbf{Z}_{\text{test}} \in \mathbb{R}^{q\times d}$ with conditional mean futures $\mathbf{Z}_{\text{test}}\Theta_\star$. With the retained indices and test representations held fixed, define

$$\mathcal{R}(\widehat{\Theta}) = \frac{1}{q}\mathbb{E}\left[\left\|\mathbf{Z}_{\text{test}}(\widehat{\Theta} - \Theta_\star)\right\|_F^2 \mid \mathbf{Z}\right]. \tag{22}$$

Theorem A.1 (Effect on a linear student). *Under the preceding assumptions, let* $\mathbf{W} = \mathbf{Z}_{\text{test}}\mathbf{A}^\dagger\mathbf{S}_{\mathcal{I}}$. *The two students have the same prediction bias, and their expected prediction errors satisfy*

$$\mathcal{R}(\widehat{\Theta}_{\text{obs}}) - \mathcal{R}(\widehat{\Theta}_{\text{proj}}) = \frac{\tau^2}{q}\|\mathbf{W}(\mathbf{I}_n - \mathbf{P}_Z)\|_F^2 \geq 0. \tag{23}$$

Proof. Using $\mathbf{S}_{\mathcal{I}}\mathbf{M} = \mathbf{A}\Theta_\star$ and $\mathbf{P}_Z\mathbf{M} = \mathbf{M}$, the two estimators can be written as follows:

$$\begin{aligned}\widehat{\Theta}_{\text{obs}} &= \mathbf{A}^\dagger\mathbf{A}\Theta_\star + \mathbf{A}^\dagger\mathbf{S}_{\mathcal{I}}\mathbf{E},\\ \widehat{\Theta}_{\text{proj}} &= \mathbf{A}^\dagger\mathbf{A}\Theta_\star + \mathbf{A}^\dagger\mathbf{S}_{\mathcal{I}}\mathbf{P}_Z\mathbf{E}.\end{aligned} \tag{24}$$

Since $\mathbb{E}[\mathbf{E} \mid \mathbf{Z}] = \mathbf{0}$, both have conditional expectation $\mathbf{A}^\dagger\mathbf{A}\Theta_\star$. Their prediction errors therefore share the same bias term

$$\mathbf{B} = \mathbf{Z}_{\text{test}}(\mathbf{A}^\dagger\mathbf{A} - \mathbf{I}_d)\Theta_\star. \tag{25}$$

Their zero-mean random prediction components are $\mathbf{W}\mathbf{E}$ and $\mathbf{W}\mathbf{P}_Z\mathbf{E}$, respectively. Using $\mathbb{E}[\mathbf{E}\mathbf{E}^\top \mid \mathbf{Z}] = \tau^2\mathbf{I}_n$ gives:

$$\begin{aligned}\mathcal{R}(\widehat{\Theta}_{\text{obs}}) &= \frac{1}{q}\|\mathbf{B}\|_F^2 + \frac{\tau^2}{q}\|\mathbf{W}\|_F^2,\\ \mathcal{R}(\widehat{\Theta}_{\text{proj}}) &= \frac{1}{q}\|\mathbf{B}\|_F^2 + \frac{\tau^2}{q}\|\mathbf{W}\mathbf{P}_Z\|_F^2.\end{aligned} \tag{26}$$

Because $\mathbf{P}_Z$ is an orthogonal projector,

$$\|\mathbf{W}\|_F^2 = \|\mathbf{W}\mathbf{P}_Z\|_F^2 + \|\mathbf{W}(\mathbf{I}_n - \mathbf{P}_Z)\|_F^2. \tag{27}$$

Subtracting the two risks proves Equation (23). □

Theorem A.1 shows that PTC targets preserve prediction bias while reducing or preserving prediction variance under the stated assumptions. This guarantee complements the residual-gradient analysis in Section 3.4 by connecting target projection to downstream prediction error in a linear setting. The experiments in Section 4 further demonstrate the practical effectiveness of target projection across datasets and student architectures, including nonlinear students beyond the scope of this guarantee.

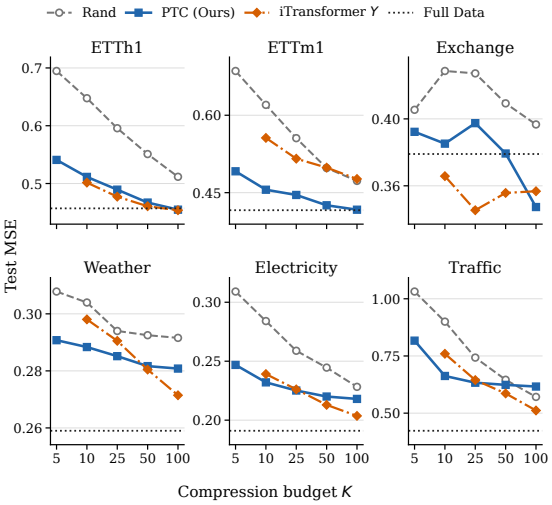


Figure 8: Test MSE across condensation budgets with iTransformer students. Full-data training is included as a reference.

B ADDITIONAL EXPERIMENTS

B.1 Sensitivity Study across Condensation Budgets

We further examine the effect of target construction across condensation budgets using iTransformer students. For each budget, all variants use identical randomly selected historical windows and differ only in their targets. Figure 8 reports the comparison, with full-data training included as a reference. The iTransformer-generated targets are evaluated at $K \in \{10, 25, 50, 100\}$.

The results reveal a budget-dependent trade-off in target construction. On Weather, Electricity, and Traffic, linear target projection in PTC yields lower MSE than iTransformer-generated targets at $K = 10$ and $K = 25$, but the ordering reverses at larger budgets. As more windows are retained, the improvement from linear projection becomes limited, whereas iTransformer-generated targets continue to improve forecasting performance. On Traffic at $K = 100$, the latter reduce MSE from 0.616 to 0.512, and even the observed targets outperform the linearly projected targets. This pattern suggests that, although linear projection provides effective supervision under small budgets, it may discard predictive structure that becomes useful as the condensed dataset grows. A more expressive target generator can potentially retain such structure and allow nonlinear students to benefit more from additional windows. This advantage remains dataset-dependent, as linear projection achieves lower MSE on ETTm1 across all evaluated budgets. The comparison with Figure 5 also suggests that the effectiveness of linear target projection depends on the downstream student. While the projected targets bring DLinear close to full-data performance on most datasets, more expressive target generators can help iTransformer narrow the remaining gap at larger budgets. Target construction should therefore account for both the condensation budget and the downstream architecture.

B.2 Evaluation on LargeST

We further evaluate PTC on LargeST-CA[3], a substantially larger traffic dataset containing 35,040 timesteps and 8,600 variables for 2019 after aggregation to 15-minute intervals. This amounts to approximately 301 million scalar observations, about 20 times the number in the Traffic benchmark used in our main experiments. We use chronological training, validation, and test splits with a ratio of 6/2/2, global standardization fitted on the training split, and $L = H = 96$. We compare the default linear implementation with random selection at $K \in \{5, 10, 25, 50, 100\}$ using DLinear and PatchTST students on an NVIDIA A10 GPU. Condensed-data results are averaged over three student seeds, with identical initial weights across methods for each seed, while full-data training uses one seed. The PatchTST full-data run reaches the 24-hour limit after completing 20 epochs. Its reported accuracy uses the checkpoint with the lowest validation MSE, obtained at epoch 18. Training-time measurements exclude validation and test evaluation as well as dataset construction, which takes approximately 11.2 seconds for PTC at $K = 25$.

[3]https://github.com/liuxu77/LargeST

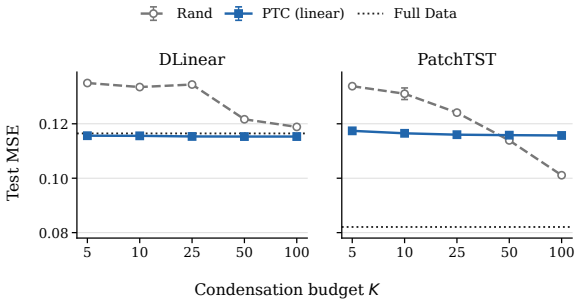


Figure 9: Test MSE on LargeST-CA with DLinear and PatchTST students across condensation budgets. Full-data training is included as a reference.

Figure 9 shows that the effectiveness of linear target projection differs between the two students. With DLinear, the projected targets outperform random selection across all budgets and achieve MSE better than the full-data reference. With PatchTST, linear projection remains effective at small budgets, but its benefit becomes limited as more windows are retained. At $K = 100$, training with observed targets achieves an MSE of 0.1011, compared with 0.1157 for linearly projected targets and 0.0821 for the full-data reference. The similar errors obtained by DLinear and PatchTST with projected targets suggest that linear target construction may restrict the additional predictive capacity of the nonlinear student. This finding complements the budget-dependent results in Appendix B.1.

Table 6 compares student-training time at $K = 25$. PTC constructs the condensed dataset in approximately 11.2 seconds and reduces DLinear training time from 9.39 to 0.64 minutes, while maintaining forecasting accuracy close to the full-data reference. For PatchTST, condensed-data training takes 33.98 minutes, compared with 1192.39 minutes for the 20 completed full-data epochs. These results demonstrate inexpensive dataset construction and substantial reductions in student-training computation on LargeST, although the accuracy gap for PatchTST indicates that these savings can come at the expense of predictive performance with the default linear targets.

Table 6: Student-training time on LargeST-CA at $K = 25$, in minutes. Condensed datasets are constructed by PTC, with training time averaged over three seeds. Full-data training uses one seed. Evaluation time are excluded.

Student	Full-data training	Condensed-data training
DLinear	9.39	0.64
PatchTST	1192.39*	33.98

*Training time for the 20 completed epochs before the 24-hour timeout.